

SAP BTP CAP Application

Development Standards & Naming Conventions

Node.js (CAP for Node) · SAP HANA Cloud · Cloud Foundry Runtime

Field	Value
Document title	SAP BTP CAP Development Standards & Naming Conventions
Client / programme	Blackmores Australia — SAP BTP Extension Programme
Applies to	All CAP Node.js services deployed to BTP Cloud Foundry (DEV / QAS / PRD)
Version	1.0
Status	Baseline — for technical lead review and project adoption
Date issued	15 September 2026
Document owner	Technical Lead / Solution Architect, BTP Extension Programme
Audience	Full-stack developers, technical leads, reviewers, DevOps engineers, QA
Review cycle	Quarterly, or on any material change to the CAP / BTP baseline

Contents

1. Purpose, Scope and Enforcement

1.1 Purpose · 1.2 Scope (in scope, out of scope, assumed landscape) · 1.3 Enforcement (normative language, enforcement statement, waivers)

2. Reference Architecture

3. Repository and Project Structure

4. Naming Conventions

4.1 Global principles · 4.2 Case convention by artefact type · 4.3 Namespaces · 4.4 Entities and elements · 4.5 Aspects, types and enums · 4.6 Associations and compositions · 4.7 Services, projections and operations · 4.8 Files and folders · 4.9 SAP HANA Cloud runtime artefacts · 4.10 BTP and Cloud Foundry resource names · 4.11 Environment variables · 4.12 Git branches, commits, tags and versions

5. Domain Modelling Standards

6. Service Layer Standards

7. Custom Logic and Event Handlers

8. Validation, Errors and Messages

9. Security and Authorisation

10. Integration Standards

11. SAP HANA Cloud Persistence Standards

12. Configuration, Profiles and Secrets

13. Logging and Observability

14. Performance and Resilience

15. Testing Standards

16. Code Quality, Linting and Formatting

17. Git Workflow and Versioning

18. CI/CD — Azure DevOps

19. CI/CD — GitHub Actions

20. Deployment and Promotion

21. Documentation and Definition of Done

22. Anti-Pattern Catalogue

22.1 Modelling · 22.2 Service and logic · 22.3 Security · 22.4 Integration · 22.5 Delivery and DevOps

Appendix A — Naming Quick Reference

Appendix B — Pull Request Review Checklist

Appendix C — Document Control

1. Purpose, Scope and Enforcement

1.1 Purpose

This document defines the mandatory engineering standard for all SAP Cloud Application Programming Model (CAP) applications built for Blackmores Australia on the SAP Business Technology Platform (BTP) Cloud Foundry runtime. It exists to make the codebase predictable: any developer joining the programme should be able to open a repository, infer the intent of every artefact from its name, and locate any piece of logic without asking the original author. It does this by:

- **Removing avoidable variance.** Where CAP offers several legitimate ways to achieve the same outcome, this document names the one the programme uses, so review effort goes on business correctness rather than style debates.
- **Making naming self-documenting.** One convention across CDS models, services, HANA artefacts, BTP service instances, Git branches and Cloud Foundry applications means the name alone tells you the layer, the domain and the owner.
- **Protecting the non-functional characteristics.** Security, resilience on S/4HANA calls, observability and upgradeability are design properties, not late-stage fixes; the rules below encode them.
- **Making the build gate objective and the solution supportable.** Every "MUST" is intended to be enforceable by a linter, a test or a pipeline check rather than by reviewer memory — and the same conventions shorten mean-time-to-diagnosis in production, where most of a solution's lifetime cost sits.

INTENT This is a standard, not a tutorial. It assumes working knowledge of CAP, CDS, OData V4, Node.js, and Cloud Foundry. Where SAP documentation already explains a mechanism, this document only states the choice the programme has made.

1.2 Scope

In scope

Area	What is covered
Runtime and stack	CAP for Node.js (@sap/cds), Node.js LTS, OData V4 service protocol, SAP HANA Cloud persistence, SAP BTP Cloud Foundry runtime.
Repository layout	Monorepo structure, MTA module decomposition, db/ and srv/ layering, shared library packaging.
Domain modelling	CDS namespaces, entities, elements, aspects, types, enums, code lists, associations and compositions, annotations.
Service layer	Service definitions, projections, actions, functions, events, draft handling, service-level annotations.
Custom logic	Event handler registration and placement, cds.q1 usage, transaction boundaries, reuse via services rather than shared mutable state.
Security	XSUAA scopes and role templates, @requires / @restrict, instance-based authorisation, attribute-based filtering, secret handling.

Integration	S/4HANA consumption through BTP destinations, external service import, SAP Event Mesh messaging, resilience patterns.
Persistence	SAP HANA Cloud HDI artefacts, schema evolution rules, table data (CSV) loading, indexing and sizing guidance.
Quality	Unit, integration and contract testing; coverage thresholds; linting; formatting; static analysis.
DevOps	Git branching and commit conventions, semantic versioning, Azure DevOps and GitHub Actions pipeline definitions, MTA build, promotion across DEV / QAS / PRD, blue-green deployment.
Anti-patterns	A catalogue of the specific practices that will cause a pull request to be rejected.

Out of scope

The following are deliberately excluded and governed elsewhere:

- **UI layer standards** (SAPUI5 / Fiori Elements, UI annotations, Launchpad content). This edition is backend only — the CAP service and SAP HANA Cloud layers. A companion UI standard is maintained separately.
- **ABAP and S/4HANA in-system development** (CDS views, RAP objects, API release), and SAP Integration Suite, Datasphere or analytics modelling. This document covers only how BTP **consumes** S/4HANA APIs.
- **BTP account-level governance** — subaccount creation, entitlements and quotas, IAS tenant configuration, corporate identity provider federation. Owned by the BTP platform team.
- **Commercial, contractual and data-privacy compliance decisions.** Any question touching personal data, cross-border transfer or retention must be raised with the Blackmores Data Privacy Officer and the programme Compliance lead; this document does not constitute legal or regulatory advice.
- **Kyma / Kubernetes runtime.** If a workload moves to Kyma this standard must be re-baselined: the CDS and naming sections remain valid, the deployment sections do not.

Assumed landscape

The standard assumes the following baseline. Where a value is environment-specific it is shown as a placeholder and must be confirmed against the programme's BTP landscape register before use (see Appendix C.2).

Landscape	Purpose	Subaccount / space	Data	Deployment trigger
DEV	Feature development, developer integration testing	blkm-btp-dev / dev	Synthetic and masked extracts only	Automatic on merge to develop
QAS	System integration test, UAT, performance test	blkm-btp-qas / qas	Masked production-like volumes	Automatic on merge to main
PRD	Production	blkm-btp-prd / prd	Live business data	Manual approval gate on a signed release tag

- System of record is **SAP S/4HANA**, consumed exclusively through released OData or SOAP APIs exposed via BTP destinations. Direct database access to S/4HANA is prohibited.
- Authentication and authorisation use **XSUAA** with SAP Cloud Identity Services (IAS) as the trusted identity provider, federated to the Blackmores corporate directory.
- Persistence is **SAP HANA Cloud**, one HDI container per CAP application per space. SQLite is used **only** for local development and automated tests.
- Applications are packaged and deployed as **Multi-Target Applications (MTA)**. `cf push` is not a supported deployment path for any landscape.

1.3 Enforcement

Normative language

Keyword	Meaning	Consequence of non-compliance
MUST / MUST NOT	Absolute requirement. Non-negotiable without a recorded waiver.	Pull request is rejected. Where automated, the pipeline fails and the build is not promotable.
SHOULD / SHOULD NOT	Strong recommendation. Deviation is permitted only with a justification recorded in the pull request description.	Reviewer raises a blocking comment; merge requires explicit technical-lead sign-off.
MAY	Genuinely optional. Both choices are acceptable.	None.

Enforcement statement

Compliance with this standard is a condition of merge, not a matter of preference. It is enforced at four points in the delivery chain, from cheapest to most expensive:

1. **Local, pre-commit.** A Husky pre-commit hook runs `cds-lint`, ESLint, and Prettier in check mode over staged files. A commit that fails is not created. A commit message that violates the Conventional Commits format is rejected by `commitlint`.
2. **Pull request, automated.** The CI pipeline runs lint, unit tests, integration tests, coverage thresholds, an `npm audit` dependency scan, a secret scan, and an `mbt build` of the full MTA. All checks are required status checks on the protected branch; none may be bypassed by a developer.
3. **Pull request, human.** At least one reviewer other than the author must approve, using the review checklist in Appendix B. Changes touching authorisation annotations, `xs-security.json`, destination configuration, or database schema require review by the technical lead **and** a second reviewer.
4. **Promotion gate.** Promotion to PRD requires a signed release tag, a green QAS pipeline run against the identical MTA archive, a reviewed HDI deployment plan for any schema change, and recorded approval in the change record. The artefact promoted to PRD **MUST** be the same `.mtar` that was tested in QAS — rebuilding for production is prohibited.

Waivers and maintenance

A **MUST** rule can be waived only by the programme technical lead. A waiver **MUST** be recorded as an ADR in `docs/decisions/` stating the rule waived, the reason, the compensating control and an expiry date, and **MUST** be referenced from a `// STANDARD-WAIVER: ADR-nnn` comment at the point in the code where the deviation occurs. Undocumented deviations are defects and are tracked as such.

This document is versioned alongside the code it governs, under `docs/standards/`, and changes follow the same pull-request and review process. A tightened rule applies to new code immediately and to existing code at the next planned touch of the affected module; retrospective mass refactoring is required only where a security or data-integrity rule is involved, in which case it is remediated as a defect.

2. Reference Architecture

2.1 Logical components

Every CAP application in the programme is composed of the same set of logical components. Naming, layering, and deployment rules in the rest of this document all key off this decomposition.

Component	MTA module type	Responsibility	Must not
CAP service	Node.js (node.js)	OData V4 endpoints, domain logic, validation, orchestration of external calls, event publication.	Contain UI assets, contain SQL DDL, hold credentials.
Database deployer	HDB (hdb)	Deploys generated HDI artefacts and CSV reference data into the HANA Cloud HDI container.	Contain hand-written artefacts that the CDS model could generate.
XSUAA instance	Managed service	Scopes, attributes, role templates, role collections; token issuance.	Contain business logic or environment-specific URLs hardcoded.
Destination service	Managed service	Resolves logical S/4HANA and third-party endpoints per landscape.	Be bypassed by hardcoded hostnames in code.
Application / Cloud Logging	Managed service	Structured log ingestion and correlation.	Receive unredacted personal data or secrets.
Event Mesh (where used)	Managed service	Asynchronous domain event publication and subscription.	Be used for synchronous request/response patterns.

2.2 Layering rule

The programme uses the standard CAP three-layer separation, and the dependency direction is one-way. A lower layer **MUST NOT** know about a higher one.

Dependency direction: srv/ imports db/; db/ never imports srv/

```
db/          Domain model – entities, aspects, types, code lists. No service concerns.
  ^
srv/         Service layer – projections, actions, events, authorisation, handlers.
  ^
app/         Consumers    – UI apps, API clients (out of scope in this edition).
```

RULE A `.cds` file under `db/` **MUST NOT** reference any artefact defined under `srv/`, and **MUST NOT** carry OData-, UI-, or authorisation-specific annotations. Those belong on the service projection.

2.3 Local development topology

Developers MUST be able to run the full service locally with no BTP connectivity, using SQLite for persistence, mocked authentication, and mocked external services. This is a hard requirement: a service that can only be exercised by deploying to DEV cannot be unit tested, and slows every subsequent change.

Terminal — local and hybrid development modes

```
# Local run with in-memory SQLite, mocked auth and mocked externals
cds watch --profile development

# Bind to real BTP services when integration testing is genuinely required
cds bind --to blkm-btp-order-db,blkm-btp-order-uaa --profile hybrid
cds watch --profile hybrid
```

3. Repository and Project Structure

3.1 Canonical structure

Every CAP repository MUST follow the structure below. Directory names are fixed; only the domain-specific file names vary. Deviating from this layout breaks the `cds` build conventions and the shared pipeline templates.

Canonical repository layout

```
blkm-btp-<domain>/
+- .github/workflows/           # GitHub Actions workflows (if GitHub-hosted)
+- .pipelines/                 # Azure DevOps YAML templates (if ADO-hosted)
+- db/
| +- schema.cds                # Root model file: includes domain files only
| +- common.cds                # Shared aspects, types, reused code lists
| +- <domain>/
| | +- sales-orders.cds
| | +- products.cds
| +- data/                     # Reference/config CSVs loaded via hdbtabledata
| | +- com.blackmores.order-OrderStatuses.csv
| +- undeploy.json             # Explicit, reviewed drop list (see 11.4)
+- srv/
| +- sales-order-service.cds    # Service definition (projections, actions, events)
| +- sales-order-service.js     # Thin handler registration only
| +- handlers/
| | +- sales-order-validation.js
| | +- sales-order-pricing.js
| +- lib/                      # Pure, framework-agnostic helpers (unit-testable)
| | +- price-calculator.js
| +- external/                 # Imported external service definitions
| | +- API_SALES_ORDER_SRV.edmx
| | +- API_SALES_ORDER_SRV.csn
| +- _i18n/
|   +- messages.properties
|   +- messages_en_AU.properties
+- test/
| +- unit/
| +- integration/
| +- fixtures/
```

```

+- docs/
| +- standards/           # This document
| +- decisions/           # ADRs, including standard waivers
+- .cdsrc.json            # CDS configuration (profiles, features)
+- .eslintrc.json
+- .prettierrc
+- mta.yaml               # Deployment descriptor
+- dev.mtaext / qas.mtaext / prd.mtaext
+- xs-security.json       # Scopes, attributes, role templates
+- package.json
+- README.md

```

3.2 Repository granularity

One repository per **bounded context**, not one per team and not one per entity. A bounded context is a set of entities that change together and share a single owner. The practical test: if two candidate contexts can never be released independently of one another, they belong in the same repository.

- A repository **MUST** deploy as exactly one MTA. Two independently releasable applications **MUST NOT** share a repository.
- Code shared by two or more repositories **MUST** be extracted into a versioned npm package (a CAP reuse model, published to the programme's private registry), not copied and not referenced by relative path across repositories.
- A reuse package **MUST** be semantically versioned and consumed by caret range (^1.4.0) only for patch and minor; a major upgrade **MUST** be a deliberate, separately tested change.

3.3 package.json requirements

package.json — mandatory fields and scripts

```

{
  "name": "blkm-btp-order",
  "version": "1.4.0",
  "private": true,
  "engines": { "node": "^22.0.0" },
  "dependencies": {
    "@sap/cds": "^8.6.0",
    "@sap/xssec": "^4.2.4",
    "@sap-cloud-sdk/http-client": "^3.24.0",
    "@cap-js/hana": "^1.5.0",
    "express": "^4.21.2"
  },
  "devDependencies": {

```

```

"@cap-js/sqlite": "^1.10.0",
"@sap/cds-dk": "^8.6.0",
"@sap/eslint-plugin-cds": "^3.1.0",
"jest": "^29.7.0",
"husky": "^9.1.7",
"@commitlint/cli": "^19.6.1"
},
"scripts": {
  "start": "cds-serve",
  "watch": "cds watch --profile development",
  "lint": "cds-lint . && eslint . --max-warnings 0",
  "format:check": "prettier --check \"**/*.js,json,md,cds\"",
  "test": "jest --coverage --runInBand",
  "test:unit": "jest test/unit",
  "build": "mbt build -p=cf --mtar=blkm-btp-order.mtar",
  "prepare": "husky"
}
}

```

- The `engines.node` range MUST be pinned to the Node.js LTS major that the Cloud Foundry Node.js buildpack provides in the target landscape. A mismatch between local and buildpack Node versions is a recurring source of "works on my machine" defects.
- `@sap/cds` MUST be constrained to a caret range on a single major version. Wildcards (`*`, `latest`, `x`) are prohibited.
- `package-lock.json` MUST be committed, and CI MUST install with `npm ci` — never `npm install` — so the pipeline builds exactly what was tested locally.
- `@sap/cds-dk` and all test tooling MUST be `devDependencies` only. Shipping development tooling into the Cloud Foundry droplet inflates the deployment and widens the vulnerability surface.
- The `scripts` block MUST expose `lint`, `test`, and `build` with these exact names, because the shared pipeline templates invoke them by name.

4. Naming Conventions

Naming is the highest-leverage convention in the document because it is the one a developer encounters hundreds of times a day. The rules below are exhaustive for the backend layers in scope. Appendix A condenses them into a one-page reference.

4.1 Global principles

- **English, unabbreviated, business vocabulary.** Use the term the business uses. `PurchaseOrder`, not `PO`, `PurchOrd`, or a non-English equivalent.
- **No Hungarian notation and no type suffixes.** `strName`, `tblOrders`, `OrderObject` are all prohibited. The type is visible from the declaration.
- **No redundant qualification.** Inside entity `Orders`, an element is `status`, not `orderStatus`. The entity already supplies the context.
- **Abbreviations only from the approved list.** The only abbreviations permitted are those in Appendix A.4 (ID, URL, UUID, API, VAT, ABN, SKU, EAN, GTIN, UOM, ETA). Anything else is spelled out.
- **Names describe what a thing is, never when it was written or who wrote it.** `OrderServiceV2`, `OrderServiceNew`, `OrderService_MT` are prohibited; version through Git, not through names.
- **Never encode the landscape into an application-internal name.** Environment differentiation happens in MTA extension descriptors and destinations, never in CDS or JavaScript identifiers.

4.2 Case convention by artefact type

Artefact	Case	Example
CDS namespace	lower.dot.case	<code>com.blackmores.order</code>
CDS entity	PascalCase, plural	<code>SalesOrders</code> , <code>Products</code>
CDS element / field	camelCase, singular	<code>grossAmount</code> , <code>deliveryDate</code>
CDS type / aspect	PascalCase, singular	<code>Amount</code> , <code>Auditable</code>
CDS enum type	PascalCase singular; values UPPER_SNAKE	<code>OrderStatus</code> with <code>OPEN</code> , <code>IN_DELIVERY</code>
To-one association	camelCase, singular	<code>customer</code> , <code>shipToParty</code>
To-many assoc. / composition	camelCase, plural	<code>items</code> , <code>attachments</code>
CDS service	PascalCase ending <code>Service</code>	<code>SalesOrderService</code>
Action / function	camelCase verb phrase	<code>confirmOrder</code> , <code>calculateFreight</code>

Event	PascalCase past-tense noun phrase	OrderConfirmed, StockReserved
.cds / .js file	kebab-case	sales-order-service.cds
JS variable / function	camelCase	grossAmount, resolveCustomer
JS class	PascalCase	PriceCalculator
JS constant (module-level)	UPPER_SNAKE_CASE	MAX_RETRY_ATTEMPTS
Environment variable	UPPER_SNAKE_CASE	S4_REQUEST_TIMEOUT_MS
HANA runtime object	UPPER_SNAKE (CDS-generated)	COM_BLACKMORES_ORDER_SALESORDERS
XSUA scope	PascalCase after \$XSAPPNAME.	\$XSAPPNAME.OrderWrite
CF application	kebab-case with prefix	blkm-btp-order-srv
Git branch	kebab-case with type prefix	feature/BLK-1234-order-split

4.3 Namespaces

Namespaces prevent collision once reuse models and external services enter the model. The programme uses a three-segment scheme.

Namespace pattern

```
namespace com.blackmores.<context>;           // e.g. com.blackmores.order
namespace com.blackmores.<context>.<sub>;      // only when a context genuinely needs sub-domains
```

- Segment 1–2 (com.blackmores) is fixed for all programme-owned models.
- Segment 3 is the bounded context, singular, lower case: order, product, customer, logistics, finance.
- A fourth segment is permitted only where a context exceeds roughly 25 entities and a genuine sub-domain exists. Do not create com.blackmores.order.entities or .types — technical grouping is not a namespace concern.
- Reuse packages published to the private registry **MUST** use com.blackmores.common.<topic> so consumers can tell shared from local at a glance.
- Imported external services keep the **SAP-supplied** namespace and name unchanged (for example API_BUSINESS_PARTNER). Renaming an imported service breaks regeneration when the API is re-imported.

4.4 Entities and elements

db/order/sales-orders.cds — entity and element naming applied

```
using { cuid, managed, sap.common.CodeList } from '@sap/cds/common';
```

```

using { com.blackmores.common.Amount, com.blackmores.common.Auditable } from '../common';

namespace com.blackmores.order;

/**
 * A sales order raised against a Blackmores selling entity.
 * Source of truth is S/4HANA; this entity holds the BTP-side extension state.
 */
entity SalesOrders : cuid, managed, Auditable {
    orderNumber      : String(10) @assert.unique @title: 'Order Number';
    s4DocumentNumber : String(10) @readonly;
    orderDate        : Date @mandatory;
    requestedDate    : Date;
    status           : Association to OrderStatuses @mandatory default 'DRAFT';
    currency         : Currency;
    netAmount        : Amount;
    grossAmount      : Amount @readonly;
    customer         : Association to one Customers @mandatory;
    salesOrg         : Association to one SalesOrganisations;
    items            : Composition of many SalesOrderItems on items.parent = $self;
}

entity SalesOrderItems : cuid, managed {
    parent          : Association to one SalesOrders;
    itemNumber      : Integer @mandatory;
    product         : Association to one com.blackmores.product.Products @mandatory;
    quantity        : Decimal(13, 3) @mandatory @assert.range: [0.001, 9999999.999];
    unit            : String(3);
    netAmount       : Amount;
}

entity OrderStatuses : CodeList {
    key code : String(10) enum { DRAFT; SUBMITTED; CONFIRMED; CANCELLED };
}

```

- Entity names are **plural** because an entity is a set. SalesOrders, not SalesOrder. This also matches the OData entity-set name CAP generates, so the API and the model read identically.

- Element names are **singular** unless the element itself holds a collection.
- Boolean elements MUST read as a predicate: `isBlocked`, `hasAttachments`, `requiresApproval`. Never `blocked_flag`, `flagBlocked`, or a bare `deleted`.
- Date and time elements MUST carry the unit in the name where ambiguity is possible: `orderDate` (Date), `createdAt` (Timestamp), `validFrom` / `validTo` for intervals. A bare date is prohibited.
- Amount elements MUST be paired with a currency, and quantity elements with a unit of measure, in the same entity. A number without its unit is a defect waiting to happen.
- Fields mirrored from S/4HANA MUST be prefixed `s4` (`s4DocumentNumber`, `s4BusinessPartner`) so provenance is obvious and reviewers know the field is not authoritative locally.
- Every entity MUST carry a doc comment stating what it represents and where the source of truth lives. CAP surfaces these into the generated EDMX, so the comment doubles as API documentation.

ANTI-PATTERN Do not name an element type, class, key, new, default, self, or virtual. These collide with CDS or JavaScript reserved words and force quoting throughout the codebase.

4.5 Aspects, types and enums

db/common.cds — shared types and aspects

```
namespace com.blackmores.common;

// Reusable type: use everywhere a monetary amount appears.
type Amount : Decimal(15, 2) @Measures.ISOCurrency: currency_code;

// Reusable aspect: mixed in, never inherited from a concrete entity.
aspect Auditable {
    sourceSystem    : String(10) @readonly;
    lastSyncedAt    : Timestamp  @readonly;
    syncStatus      : String(10) @readonly;
}

// Enum type: PascalCase type, UPPER_SNAKE values, explicit string codes.
type ApprovalDecision : String(12) enum {
    APPROVED    = 'APPROVED';
    REJECTED    = 'REJECTED';
    DELEGATED   = 'DELEGATED';
}
```

- Aspect names MUST be adjectival or role-like (`Auditable`, `Versioned`, `Localised`), signalling that they are mixed in. An aspect named like a noun entity will be mistaken for an entity.
- A named type MUST be created for any scalar concept used in three or more places. Repeating `Decimal(15,2)` across twelve entities guarantees that one of them will drift.
- Enum values MUST have explicit literals. Relying on CAP's implicit value derivation makes the persisted value a side effect of the identifier name, so a later rename silently corrupts stored data.

- Where the value set is business-maintainable or needs translated descriptions, use `sap.common.CodeList` with a code-list entity instead of an enum. Enums are for closed, developer-owned sets only.

4.6 Associations and compositions

Intent	Construct	Naming	Example
Reference to an independent object	Association to one	camelCase singular, the role it plays	customer, shipToParty
Reference to many independent objects	Association to many ... on ...	camelCase plural	relatedOrders
Owned child collection (lifecycle-bound)	Composition of many ... on ...	camelCase plural, unqualified	items, attachments
Backlink on the child	Association to one <Parent>	always parent	parent : Association to one SalesOrders
Code list / value help	Association to <CodeList>	camelCase singular, the concept	status, country, currency

- Use **Composition** when the child cannot exist without the parent and must be deleted with it; use **Association** otherwise. Choosing composition for a shared master-data reference will cascade-delete master data — a severe defect.
- Managed compositions (no explicit on condition) are preferred for simple parent-child, as CAP then generates and maintains the backlink. Use an explicit on only where a non-default key relationship is genuinely required.
- The backlink element on a child entity **MUST** be called `parent`. Naming it `order`, `header`, or `salesOrder` means every generic handler and every test fixture has to special-case it.
- A to-many association **MUST NOT** be exposed on a service projection without a target-side filter or an explicit `@cds.query.limit`, or a single read can pull an unbounded result set.

4.7 Services, projections and operations

srv/sales-order-service.cds — service definition conventions

```
using { com.blackmores.order as db } from '../db/schema';

@path      : '/api/v1/sales-order'
@requires: 'authenticated-user'
service SalesOrderService {

    @odata.draft.enabled

    entity SalesOrders as projection on db.SalesOrders {

        *,
```

```

    items : redirected to SalesOrderItems
} excluding { createdBy, modifiedBy };

entity SalesOrderItems as projection on db.SalesOrderItems;

@readonly
entity OrderStatuses as projection on db.OrderStatuses;

// Bound action: verb phrase, camelCase, acts on one instance.
extend entity SalesOrders with actions {
    action confirmOrder(confirmationNote : String(200)) returns SalesOrders;
    action cancelOrder(reason : String(200) @mandatory) returns SalesOrders;
    function calculateFreight() returns Decimal(15, 2);
}

// Unbound action: only where the operation is genuinely service-scoped.
action recalculateOpenOrders(salesOrg : String(4) @mandatory) returns Integer;

// Outbound domain event: past tense, PascalCase.
event OrderConfirmed : {
    orderId      : UUID;
    orderNumber  : String(10);
    confirmedAt  : Timestamp;
};
}

```

- A service name **MUST** end in **Service** and **MUST** name the aggregate it serves, not the consumer: `SalesOrderService`, not `MobileService`, `FioriService`, or `Backend`. If two consumers need different shapes of the same data, expose two projections, not two consumer-named services.
- `@path` **MUST** be explicit, kebab-case, and versioned: `/api/v<major>/<resource>`. Relying on the generated path couples the public URL to the internal service identifier, so a refactor becomes a breaking API change.
- Service entities **MUST** be **projections** on db entities, never redefinitions. A projection makes the exposure decision explicit and auditable; `excluding` is the mechanism for withholding internal fields.
- Actions mutate and are named with an imperative verb phrase. Functions are side-effect free and read-only, and **MUST NOT** write. A "function" that writes will be cached or retried by clients and corrupt data.
- Prefer **bound** operations. An unbound action taking an ID as a parameter is almost always a bound action written the wrong way round.
- Events are named in the **past tense** because they describe something that has already happened (`OrderConfirmed`, not `ConfirmOrder`). The event payload **MUST** carry identifiers and the minimum business context — never the full entity, and never internal technical fields.

- One service per bounded context is the default. A second service is justified only by a genuinely different authorisation model, protocol, or lifecycle — for example an internal integration service alongside a user-facing one.

ANTI-PATTERN A single service `ApiService` exposing every entity in the model with `*` and no excluding clause. This publishes internal fields, defeats field-level authorisation review, and makes every future model change a potential breaking API change.

4.8 Files and folders

File kind	Convention	Example
Domain model file	kebab-case, plural, matching the aggregate	<code>db/order/sales-orders.cds</code>
Service definition	kebab-case singular aggregate + <code>-service.cds</code>	<code>srv/sales-order-service.cds</code>
Service implementation	same base name as the <code>.cds</code> , <code>.js</code> extension	<code>srv/sales-order-service.js</code>
Handler module	<code><aggregate>-<concern>.js</code>	<code>srv/handlers/sales-order-validation.js</code>
Pure helper library	kebab-case noun phrase	<code>srv/lib/price-calculator.js</code>
Unit test	mirror of the module under test + <code>.test.js</code>	<code>test/unit/price-calculator.test.js</code>
Integration test	<code><service>.integration.test.js</code>	<code>test/integration/sales-order-service.integration.test.js</code>
Test fixture	kebab-case describing the scenario	<code>test/fixtures/order-with-blocked-customer.json</code>
Reference data CSV	<code><namespace>-<Entity>.csv</code> (CAP-mandated)	<code>db/data/com.blackmores.order-OrderStatuses.csv</code>
MTA extension	<code><landscape>.mtaext</code>	<code>qas.mtaext</code>
ADR	<code>NNNN-kebab-title.md</code>	<code>docs/decisions/0007-use-event-mesh-for-stock.md</code>

- A service implementation file **MUST** share the base name of its `.cds` file. CAP resolves handlers by this convention; breaking it means the handlers silently never register — one of the hardest CAP defects to spot, because the service still starts and still answers requests.
- File names **MUST NOT** contain spaces, upper-case letters, or underscores. Cloud Foundry Linux containers are case-sensitive while Windows development machines are not; a casing mismatch passes locally and fails on deploy.

- A .js file SHOULD NOT exceed roughly 300 lines. Beyond that, split by concern into `srv/handlers/`.

4.9 SAP HANA Cloud runtime artefacts

CDS generates HDI artefact names deterministically from the fully-qualified CDS name, replacing dots with underscores and upper-casing. Developers MUST NOT rename them, and MUST NOT hand-write `.hdbtable` files for anything the model can generate.

CDS definition	Generated HDI artefact	HANA runtime object
<code>com.blackmores.order.SalesOrders</code>	<code>com.blackmores.order.SalesOrders.hdbtable</code>	<code>COM_BLACKMORES_ORDER_SALES</code>
<code>SalesOrderService.SalesOrders</code> (projection)	<code>...SalesOrderService.SalesOrders.hdbview</code>	<code>SALESORDERSERVICE_SALESORD</code>
Composition child <code>SalesOrderItems</code>	<code>com.blackmores.order.SalesOrderItems.hdbtable</code>	<code>COM_BLACKMORES_ORDER_SALES</code>

- Because runtime names are derived from the namespace, **renaming a namespace renames every table**. A namespace change after the first PRD deployment is a data migration, not a refactor, and requires an ADR plus a reviewed migration plan.
- Keep fully-qualified CDS names short enough that the generated HANA identifier stays within the 127-character limit. Deeply nested namespaces plus long entity names will breach it, and the failure appears only at HDI deploy time.
- Native HDI artefacts that genuinely cannot be modelled in CDS (for example a `.hdbprocedure`, or a calculation view) MUST live under `db/src/` and be documented in an ADR explaining why CDS was insufficient.

4.10 BTP and Cloud Foundry resource names

Platform resource names are what an on-call engineer reads first during an incident, so they MUST be unambiguous without context. The pattern is `blkm-<platform>-<context>-<role>`.

Resource	Pattern	Example
MTA ID	<code>com.blackmores.btp.<context></code>	<code>com.blackmores.btp.order</code>
CF application (service)	<code>blkm-btp-<context>-srv</code>	<code>blkm-btp-order-srv</code>
CF application (DB deployer)	<code>blkm-btp-<context>-db-deployer</code>	<code>blkm-btp-order-db-deployer</code>
HDI container service instance	<code>blkm-btp-<context>-db</code>	<code>blkm-btp-order-db</code>
XSUAA service instance	<code>blkm-btp-<context>-uaa</code>	<code>blkm-btp-order-uaa</code>

Destination service instance	blkm-btp-<context>-dest	blkm-btp-order-dest
Event Mesh instance	blkm-btp-<context>-mesh	blkm-btp-order-mesh
Logging instance	blkm-btp-<context>-logs	blkm-btp-order-logs
Destination (S/4 API)	S4_<API_NAME>	S4_API_SALES_ORDER_SRV
Role collection	BLKM_<Context><Role>	BLKM_OrderProcessor
Event Mesh topic	blkm/<context>/<entity>/<event>/v1	blkm/order/salesorder/confirmed/v1

RULE Resource names MUST NOT contain the landscape (-dev, -qas, -prd). The subaccount and space already provide that isolation; embedding it in the name forces landscape-specific `mta.yaml` files, which is exactly the problem MTA extension descriptors exist to solve.

4.11 Environment variables and configuration keys

- Environment variables MUST be UPPER_SNAKE_CASE, prefixed by the subsystem they configure: S4_REQUEST_TIMEOUT_MS, MESH_PUBLISH_RETRY_LIMIT, FEATURE_ORDER_SPLIT_ENABLED.
- Any variable expressing a duration MUST carry the unit suffix (_MS, _SECONDS, _DAYS). Unitless timeouts are misread constantly.
- Feature flags MUST be prefixed FEATURE_ and suffixed _ENABLED, MUST default to `false`, and MUST have a removal ticket raised at the time they are introduced. A flag with no removal ticket is permanent complexity.
- CDS configuration keys in `.cdsrc.json` follow the CAP-defined lower-camel structure and MUST NOT be invented ad hoc — use the documented `cds.requires`, `cds.features`, and `cds.log` trees.
- Configuration values MUST NOT be read at request time. Resolve them once at module load or in `init()` and cache them; `cds.env` lookups in a hot path are measurable overhead.

.cdsrc.json — profile-based configuration

```
{
  "requires": {
    "db": {
      "kind": "hana-cloud",
      "[development]": { "kind": "sqlite", "credentials": { "url": ":memory:" } }
    },
    "auth": {
      "kind": "xsuaa",
      "[development]": { "kind": "mocked", "users": {
        "order.processor": { "roles": ["OrderProcessor"], "attr": { "SalesOrg": ["1710"] } },
        "order.viewer": { "roles": ["OrderViewer"], "attr": { "SalesOrg": ["1710"] } }
      } }
    }
  }
}
```

```

    },
    "S4_API_SALES_ORDER_SRV": {
      "kind": "odata-v2",
      "model": "srv/external/API_SALES_ORDER_SRV",
      "credentials": {
        "destination": "S4_API_SALES_ORDER_SRV",
        "path": "/sap/opu/odata/sap/API_SALES_ORDER_SRV"
      },
      "[development]": { "credentials": { "url": "http://localhost:4004/mock" } }
    }
  },
  "log": { "service": true, "levels": { "cds": "warn", "sql": "error" } },
  "features": { "assert_integrity": true, "optimized_search": true }
}

```

4.12 Git branches, commits, tags and versions

Artefact	Pattern	Example
Long-lived branches	main (QAS-ready), develop (DEV-ready)	—
Feature branch	feature/<TICKET>-<kebab-summary>	feature/BLK-1234-order-split-logic
Bug fix branch	bugfix/<TICKET>-<kebab-summary>	bugfix/BLK-1301-null-currency
Production hotfix	hotfix/<TICKET>-<kebab-summary>	hotfix/BLK-1402-tax-rounding
Release branch	release/<semver>	release/1.5.0
Release tag	v<semver> (annotated, signed)	v1.5.0
Version	Semantic versioning on the MTA and package.json	1.5.0

Conventional Commits format — enforced by commitlint

<type>(<scope>): <imperative summary, max 72 chars>

<body: why, not what — wrap at 100 chars>

Refs: BLK-1234

BREAKING CHANGE: <only when the public API or schema changes incompatibly>

- Permitted type values: feat, fix, perf, refactor, test, docs, build, ci, chore, revert.
- Permitted scope values are the repository's bounded-context modules plus db, srv, security, deploy. The list MUST be declared in `commitlint.config.js` so it is enforced rather than remembered.
- Every commit MUST reference a work item (Refs: BLK-####). A commit with no traceable requirement cannot be assessed at release.
- Branch names MUST include the ticket ID so the pipeline can link a build to a work item automatically.
- MAJOR is incremented for an incompatible change to the OData contract or database schema, MINOR for backward-compatible capability, PATCH for fixes. The version in `package.json` and `mta.yaml` MUST match, and this is checked in CI.

5. Domain Modelling Standards

5.1 Keys and identifiers

- Every persistent entity **MUST** use the `cuid` aspect, giving a single UUID technical key. Business keys (`orderNumber`) are modelled as separate elements with `@assert.unique`.
- Composite and semantic primary keys are prohibited on new entities. They propagate into every association, every URL, and every test fixture, and any future change to the business key becomes a schema migration.
- Code-list entities are the one exception: they **MUST** key on a short, stable `String` code, because the code is the value stored on referencing entities and appears in integration payloads.
- A UUID key **MUST NOT** be exposed to users as an identifier in the UI or in integration payloads. Surface the business key; keep the UUID internal.

5.2 Managed data and auditing

Use the managed aspect; never re-implement it

```
// Correct: reuse the CAP-managed aspect. CAP fills these automatically.
entity SalesOrders : cuid, managed { ... }

// managed contributes: createdAt, createdBy, modifiedAt, modifiedBy

// Wrong: hand-rolled equivalents. These will not be maintained by the framework,
// will not be filled on draft activation, and will drift from platform behaviour.
entity SalesOrders : cuid {
    created_on    : Timestamp;
    created_user  : String(50);
}
```

- `createdBy` and `modifiedBy` **MUST** be `@readonly` on all service projections, and **SHOULD** be excluded from any projection exposed to a non-administrative consumer, as they carry user identity.
- Where a full change history is required, use `@changeLog` from the `@cap-js/change-tracking` plugin rather than writing bespoke shadow tables. Bespoke audit tables invariably miss the draft and bulk-update paths.
- Audit-relevant events (authorisation failures, data exports, configuration changes) **MUST** be written to the SAP Audit Log service, not only to application logs. Application logs are retained for days; audit logs for the period the retention policy requires.

5.3 Declarative validation first

Constraints that CDS can express **MUST** be expressed in CDS. Declarative constraints are enforced on every write path — REST, draft activation, bulk load, and internal service calls — whereas a hand-written handler only guards the paths its author remembered.

Declarative constraints in CDS

```
entity Products : cuid, managed {
    sku          : String(18)    @mandatory @assert.unique;
    ean          : String(13)    @assert.format: '^[0-9]{13}$';
}
```

```

name          : localized String(120) @mandatory;

shelfLifeDays : Integer          @assert.range: [1, 3650];

status        : Association to ProductStatuses @mandatory;

netWeight     : Decimal(11, 3) @assert.range: [0, 99999.999];

weightUnit    : String(3)       @assert.target;

}

```

Rule to express	Declarative mechanism
Field is required	@mandatory
Value must be unique	@assert.unique
Numeric or date bounds	@assert.range: [min, max]
Format / pattern	@assert.format with a regular expression
Closed value set	enum, or an association to a CodeList
Referenced record must exist	@assert.target (with cds.features.assert_integrity)
Read-only to all consumers	@readonly
Write-once	@insertonly
Multi-field or stateful rule	Custom before handler — the only legitimate case

ANTI-PATTERN Re-implementing @mandatory or range checks as `if (!req.data.sku) req.error(...)` in a before CREATE handler. It duplicates the model, produces inconsistent error payloads, and is bypassed entirely by draft activation and CSV load.

5.4 Localisation

- Any element holding user-facing descriptive text **MUST** be declared `localized`. Retrofitting `localized` after go-live requires a data migration, because CAP moves the column into a generated `.texts` entity.
- `sap.common.CodeList` **MUST** be used for all code lists; it supplies `name`, `descr`, and the `.texts` structure. Never model a code list as a plain entity with an English-only description column.
- Blackmores operates across multiple markets. The default locale for this programme is `en_AU`; `en` is the fallback. Locale-specific message bundles **MUST** use the CAP naming convention `messages_{locale}.properties`.

5.5 Model hygiene

- `db/schema.cds` MUST contain `using` statements only — no entity definitions. It is the model index, and keeping it declarative makes the model's composition visible on one screen.
- An entity that must not be persisted MUST be annotated `@cds.persistence.skip`. An entity that maps to a table managed outside the model MUST use `@cds.persistence.exists`. Silently leaving either to chance produces surprise tables in the HDI container.
- Unused entities, elements, and types MUST be deleted, not commented out. Commented-out model code is indistinguishable from work in progress, and CDS files are in Git.
- The model MUST build clean: `cds compile db srv --to sql` with zero warnings. Warnings in CDS compilation are nearly always real defects in disguise.

6. Service Layer Standards

6.1 Exposure is a deliberate decision

- A service **MUST** expose the narrowest projection that satisfies its consumers. Start from nothing and add fields, not from `*` and remove them.
- Internal, technical, and provenance fields (`s4*`, `syncStatus`, `createdBy`, `modifiedBy`) **MUST** be excluded from consumer-facing projections unless a stated requirement needs them.
- Read-only services **MUST** be annotated `@readonly` at service or entity level rather than relying on the absence of write handlers. Absence of a handler is not a control — CAP's generic handlers will happily write.
- Every service **MUST** declare `@requires` at service level. A service with no service-level `@requires` is open to any authenticated caller by default and will fail security review.

6.2 API versioning and compatibility

- The major version **MUST** appear in `@path (/api/v1/...)`. A breaking change **MUST** be delivered as a new versioned path, with the previous version retained for an agreed deprecation window.
- These changes are **breaking** and require a major version: removing or renaming an exposed element, narrowing a type or length, adding `@mandatory` to an existing input, changing an enum value, changing an operation signature, changing the cardinality of an association.
- These changes are **compatible**: adding an optional element, adding a new entity or operation, adding an annotation that does not change the payload, relaxing a constraint.
- Deprecation **MUST** be signalled in the model and the API documentation at least one release before removal, and consumers **MUST** be notified through the programme's API register.

6.3 Draft handling

Draft is a UI-oriented capability with real backend consequences; enable it deliberately.

- Enable `@odata.draft.enabled` only on entities a user genuinely edits in a multi-step, save-later flow. Draft doubles the tables, complicates every query, and changes which events fire.
- Validation intended to run at save time **MUST** be registered on the **draft activation** event, not only on `CREATE/UPDATE`, or it will never run for draft-enabled entities.
- A draft-enabled entity **MUST NOT** also be written by an integration or batch path. Mixing draft and non-draft writes on one entity produces orphaned drafts and is a frequent source of production data confusion.

srv/handlers/sales-order-validation.js — draft-aware validation

```
// Validation that must run when the user saves a draft
this.before('SAVE', SalesOrders, async (req) => {
  const { items } = req.data;
  if (!items || items.length === 0) {
    return req.error({
      code: 'ORDER_NO_ITEMS', message: 'order.noItems', target: 'items', status: 400
    });
  }
}
```

```
});
```

6.4 Pagination, filtering and projection

- Every collection-returning service entity **MUST** have an explicit page-size ceiling. Set `@cds.query.limit: { default: 50, max: 500 }` at entity or service level. Without it, a client can request every row.
- Free-text search **MUST** be restricted to indexed elements using `@cds.search: { name, sku }`. The default searches every string column, which on a large HANA table is a full scan.
- Server-driven paging **MUST** be used for any integration that reads large collections; clients **MUST** follow `@odata.nextLink` rather than incrementing `$skip` themselves.
- Aggregation for reporting **SHOULD** be pushed into the database through CDS views or `@Aggregation.ApplySupported`, never performed by reading rows into Node.js and reducing in memory.

Annotation	Purpose	Where
<code>@cds.query.limit</code>	Caps page size and sets the default	Service or entity
<code>@cds.search</code>	Restricts <code>\$search</code> to named, indexed elements	Entity
<code>@readonly</code> / <code>@insertonly</code>	Declares the permitted write operations	Entity or element
<code>@cds.persistence.skip</code>	Prevents table generation	Entity (db layer)
<code>@assert.target</code>	Enforces referential integrity on write	Association
<code>@odata.draft.enabled</code>	Enables draft editing	Service entity
<code>@Capabilities.*</code>	Declares fine-grained OData capabilities to clients	Service entity

7. Custom Logic and Event Handlers

7.1 Thin service file, handlers in modules

The service implementation file is a registration manifest. It wires events to handler functions and does nothing else. All logic lives in modules under `srv/handlers/` (CAP-aware) or `srv/lib/` (pure and framework-free).

srv/sales-order-service.js — registration only

```
const cds      = require('@sap/cds');
const validation = require('./handlers/sales-order-validation');
const pricing   = require('./handlers/sales-order-pricing');
const confirm   = require('./handlers/sales-order-confirmation');

class SalesOrderService extends cds.ApplicationService {
  async init() {
    const { SalesOrders, SalesOrderItems } = this.entities;

    // Resolve external services once, at bootstrap – never per request.
    this.s4Orders = await cds.connect.to('S4_API_SALES_ORDER_SRV');
    this.messaging = await cds.connect.to('messaging');

    // Validation
    this.before(['CREATE', 'UPDATE'], SalesOrders, validation.validateHeader);
    this.before(['CREATE', 'UPDATE'], SalesOrderItems, validation.validateItem);
    this.before('SAVE', SalesOrders, validation.validateOnSave);

    // Derivation
    this.after('READ', SalesOrders, pricing.deriveGrossAmount);

    // Actions
    this.on('confirmOrder', SalesOrders, confirm.confirmOrder);
    this.on('cancelOrder', SalesOrders, confirm.cancelOrder);

    await super.init(); // MUST be last: registers CAP generic handlers
  }
}

module.exports = SalesOrderService;
```

- `await super.init()` MUST be the **last** statement in `init()`. Calling it first registers CAP's generic handlers ahead of yours, and your `before` handlers then run after the generic persistence handler — which is almost never what was intended.
- The class name MUST match the CDS service name, and the file name MUST match the `.cds` file name, or CAP will not bind the implementation.
- External services MUST be connected in `init()` and held on the service instance. `await cds.connect.to(...)` inside a request handler re-resolves the destination on every call and adds latency plus destination-service load.
- A handler module MUST export named functions taking `(req)` or `(results, req)`. It MUST NOT export an anonymous default, because stack traces then carry no function name.

7.2 Choosing the right hook

Hook	Use for	Must not be used for
<code>before</code>	Validation, defaulting, input normalisation, authorisation checks beyond <code>@restrict</code>	Reading the persisted result — it does not exist yet
<code>on</code>	Actions, functions, fully custom handling that replaces the generic behaviour	Partial logic that still needs the generic handler — you will suppress it
<code>after</code>	Derived and virtual fields, response shaping, masking	Writing to the database — errors surface after the client already believes the call succeeded
<code>on('error')</code>	Central error translation and log enrichment	Swallowing errors
<code>cds.on('served')</code>	Cross-service wiring after all services are constructed	Per-request work

7.3 Query construction — `cds.ql` only

srv/handlers/sales-order-query.js — correct query construction

```
const cds = require('@sap/cds');
const { SalesOrders } = cds.entities('com.blackmores.order');

// CORRECT: cds.ql, bound parameters, explicit column list, explicit limit.
async function findOpenOrdersForOrg(tx, salesOrg, limit = 100) {
  return tx.run(
    SELECT.from(SalesOrders)
      .columns('ID', 'orderNumber', 'grossAmount', 'currency_code',
        o => o.customer(c => { c.ID, c.name }))
      .where({ status_code: 'SUBMITTED', salesOrg_code: salesOrg })
  )
}
```

```

        .orderBy({ orderDate: 'desc' })

        .limit(limit)

    });
}

module.exports = { findOpenOrdersForOrg };

```

Prohibited pattern

```

// PROHIBITED: string-concatenated SQL. SQL-injectable, HANA-specific,
// untestable against SQLite, and invisible to CDS authorisation.

const rows = await cds.run(

    "SELECT * FROM COM_BLACKMORES_ORDER_SALESORDERS WHERE SALESORG_CODE = '" + salesOrg + "'"

);

```

- All database access **MUST** use `cds.sql` (SELECT, INSERT, UPDATE, DELETE, UPSERT). String SQL is prohibited without an ADR-recorded waiver.
- `SELECT *` is prohibited in application code. Name the columns: it bounds the payload, survives model growth, and makes the intent reviewable.
- Every unbounded SELECT **MUST** carry `.limit()`. "The table is small" is a statement about today's data, not next year's.
- Queries **MUST** be issued on the request-scoped transaction (`req.tx / srv.tx(req)`), never on a fresh root transaction, or they will not participate in the request's atomicity and will bypass tenant and user context.
- Reads inside loops are prohibited. Collect the keys and issue one `SELECT ... where ID in [...]`. The N+1 pattern is the single most common cause of slow CAP services.

ANTI-PATTERN Iterating a collection and awaiting a database read or an S/4HANA call per element. With 500 items and a 40 ms round trip this is 20 seconds of wall time, and it will pass every test that uses three fixture rows.

7.4 Transactions

- CAP manages one transaction per request. Handlers **MUST** use it and **MUST NOT** open their own root transaction with `cds.tx()` inside a request.
- A handler **MUST NOT** call `commit()` or `rollback()` on the request transaction. Throwing or calling `req.error()` rolls back; returning normally commits.
- Work that must survive a request rollback — publishing an event, writing an audit entry — **MUST** use the outbox (`cds.outboxed`) so it is transactionally enqueued and delivered after commit.
- Long-running work **MUST NOT** be performed inside a request transaction. Hand it to a background job or an Event Mesh consumer; a transaction held open across an external HTTP call will exhaust the HANA connection pool under load.

Transactional outbox for event publication

```

// Publish a domain event transactionally: enqueued with the commit,
// delivered afterwards, retried by the framework on failure.

const messaging = await cds.connect.to('messaging');

await cds.outboxed(messaging).emit('blkm/order/salesorder/confirmed/v1', {

```

```
orderId:    order.ID,  
  
orderNumber: order.orderNumber,  
  
confirmedAt: new Date().toISOString()  
});
```

7.5 Handler code style

- Handlers **MUST** be `async` and **MUST** `await` every promise. A floating promise in a handler produces an unhandled rejection that CAP cannot attribute to a request.
- A handler **SHOULD NOT** exceed roughly 50 lines. Longer means it is doing more than one thing; extract into `srv/lib/`.
- Business calculations **MUST** live in pure functions in `srv/lib/`, taking plain objects and returning plain objects, with no `req`, no `cds`, and no I/O. This is what makes them unit-testable in milliseconds.
- Module-level mutable state is prohibited. Cloud Foundry runs multiple instances and Node.js serves concurrent requests on one instance; module state is a cross-request data leak and a correctness bug.
- `console.log` is prohibited. Use the CAP logger (section 13).

8. Validation, Errors and Messages

8.1 Error contract

Every error returned to a consumer MUST be actionable, localised, and free of internal detail. Use `req.error()` for business errors and `req.reject()` only where processing must stop immediately.

srv/handlers/sales-order-validation.js — structured, accumulated errors

```
// Collect all validation failures, then let CAP return them together.
// req.error() accumulates; req.reject() throws at once.

function validateItem(req) {
  const { quantity, product_ID, unit } = req.data;

  if (quantity <= 0) {
    req.error({ code: 'ITEM_QTY_INVALID', message: 'item.quantityMustBePositive',
               target: 'quantity', status: 400 });
  }

  if (!product_ID) {
    req.error({ code: 'ITEM_PRODUCT_REQUIRED', message: 'item.productRequired',
               target: 'product_ID', status: 400 });
  }

  if (unit && !ALLOWED_UNITS.has(unit)) {
    req.error({ code: 'ITEM_UOM_INVALID',
               message: { id: 'item.unitNotAllowed', values: [unit] },
               target: 'unit', status: 400 });
  }
}
```

Field	Requirement
code	MUST be present. UPPER_SNAKE_CASE, stable, unique within the service, and never changed once released — consumers branch on it.
message	MUST be an i18n key from <code>srv/_i18n/messages.properties</code> , never a literal English string.
target	MUST be set for field-level validation failures so the consumer can highlight the offending field.
status	MUST be an accurate HTTP status: 400 bad input, 403 not permitted, 404 not found, 409 state conflict, 412 precondition, 422 business-rule rejection.

Internals	MUST NOT contain SQL text, stack traces, table or column names, hostnames, internal IDs, or any personal data.
-----------	--

8.2 Message bundles

srv/_i18n/messages.properties

```
# srv/_i18n/messages.properties (fallback – en)

item.quantityMustBePositive = Quantity must be greater than zero.

item.productRequired        = A product is required for every order item.

item.unitNotAllowed         = Unit of measure "{0}" is not permitted for this product.

order.noItems               = An order must contain at least one item before it can be saved.

order.customerBlocked       = Customer {0} is blocked for new orders. Contact Credit Control.

order.aboveUserLimit        = This order exceeds your confirmation limit of {0}. Route it for approval.

error.internal              = An unexpected error occurred. Quote reference {0} when reporting it.
```

- Keys MUST be <domain>.<camelCaseCondition> — flat, two-segment, lower-camel after the dot.
- Placeholders MUST be positional ({0}, {1}) and MUST be passed via message: { id, values }. String interpolation into the message text defeats translation.
- Messages MUST be written for the person reading them: state what is wrong and what to do about it. "Validation failed" is not an acceptable message.
- A message MUST NOT include a customer name, email address, or any other personal data beyond the minimum identifier needed to act. Error messages are logged and forwarded.
- Every new key MUST be added to the fallback bundle in the same commit. A missing key surfaces as the raw key in the UI.

8.3 Central error translation

Central error handler

```
// Translate infrastructure failures into a safe, consistent contract
// and make sure the correlation ID reaches the consumer.

this.on('error', (err, req) => {
  const log = cds.log('sales-order');
  log.error({
    code: err.code, status: err.status,
    correlationId: req.headers['x-correlation-id']
  }, err.message);

  // Never leak internals
  if (!err.code || err.status >= 500) {
    err.code = 'INTERNAL_ERROR';
  }
});
```

```
    err.message = 'error.internal';  
  
    delete err.stack;  
  
    delete err.query;  
  
    }  
});
```

9. Security and Authorisation

9.1 Principles

- **Deny by default.** Every service declares `@requires`. Every entity that is not universally readable declares `@restrict`.
- **Declare, do not code.** Authorisation **MUST** be expressed in CDS annotations so it is visible in one place and enforced on every access path. Imperative permission checks in handlers are permitted only for rules CDS cannot express, and **MUST** be accompanied by a comment explaining why.
- **Scopes are technical, roles are business.** Code and annotations reference scopes; administrators assign role collections. Never reference a role collection name from code.
- **No secret ever enters Git.** Credentials come from service bindings, destinations, or the credential store — never from source, never from `.env`, never from a pipeline variable that is not marked secret.

9.2 Scopes, attributes and role templates

xs-security.json

```
{
  "xsappname": "blkm-btp-order",
  "tenant-mode": "dedicated",
  "scopes": [
    { "name": "$XSAPPNAME.OrderRead", "description": "Read sales orders" },
    { "name": "$XSAPPNAME.OrderWrite", "description": "Create and change sales orders" },
    { "name": "$XSAPPNAME.OrderConfirm", "description": "Confirm sales orders" },
    { "name": "$XSAPPNAME.OrderAdmin", "description": "Administer order configuration" }
  ],
  "attributes": [
    { "name": "SalesOrg", "description": "Permitted sales organisations",
      "valueType": "s", "valueRequired": true },
    { "name": "OrderValueLimit", "description": "Maximum order value the user may confirm",
      "valueType": "int", "valueRequired": false }
  ],
  "role-templates": [
    { "name": "OrderViewer", "description": "Read-only access, restricted by sales org",
      "scope-references": [ "$XSAPPNAME.OrderRead" ],
      "attribute-references": [ "SalesOrg" ] },
    { "name": "OrderProcessor", "description": "Maintain and confirm orders for own sales orgs",
      "scope-references": [ "$XSAPPNAME.OrderRead", "$XSAPPNAME.OrderWrite",
                           "$XSAPPNAME.OrderConfirm" ],
      "attribute-references": [ "SalesOrg", "OrderValueLimit" ] },
  ]
}
```

```

{ "name": "OrderAdministrator", "description": "Full administrative access",
  "scope-references": [ "$XSAPPNAME.OrderRead", "$XSAPPNAME.OrderWrite",
                        "$XSAPPNAME.OrderConfirm", "$XSAPPNAME.OrderAdmin" ] }
],
"role-collections": [
  { "name": "BLKM_OrderProcessor", "description": "Blackmores order processor",
    "role-template-references": [ "$XSAPPNAME.OrderProcessor" ] }
]
}

```

- Scope names MUST be <Domain><Action> in PascalCase: OrderRead, OrderWrite, OrderConfirm, OrderAdmin. One scope per distinct capability — not one scope per entity, and not a single OrderAccess scope covering everything.
- Attribute names MUST be PascalCase and MUST mirror the CDS element they filter on, so the @restrict condition reads obviously.
- Role templates MUST be named for the **job**, not the permission set: OrderProcessor, OrderViewer, WarehouseSupervisor.
- Adding a scope is a **breaking change for administrators**: existing role collections do not pick it up automatically. Any scope change MUST be called out in the release notes with the role-collection action required, per landscape.
- xs-security.json MUST be reviewed by the technical lead on every change. It is the single highest-impact file in the repository.

9.3 Instance-based and attribute-based restrictions

srv/sales-order-service.cds — @restrict applied

```

service SalesOrderService @(requires: 'authenticated-user') {

  @restrict: [

    // Read: only orders in the user's permitted sales organisations
    { grant: 'READ', to: 'OrderRead',
      where: 'salesOrg_code = $user.SalesOrg' },

    // Write: same sales-org restriction, and only while still editable
    { grant: ['CREATE','UPDATE'], to: 'OrderWrite',
      where: "salesOrg_code = $user.SalesOrg and status_code in ('DRAFT','SUBMITTED') " },

    // Delete: drafts only, and only the user's own
    { grant: 'DELETE', to: 'OrderWrite',
      where: "createdBy = $user and status_code = 'DRAFT' " },
  ]
}

```

```

// Confirm: separate scope, further value-limited in code by user attribute
{ grant: 'confirmOrder', to: 'OrderConfirm',
  where: 'salesOrg_code = $user.SalesOrg' },

{ grant: '*', to: 'OrderAdmin' }
]

entity SalesOrders as projection on db.SalesOrders excluding { createdBy, modifiedBy };

@readonly @requires: 'OrderRead'
entity OrderStatuses as projection on db.OrderStatuses;
}

```

- The where clause of `@restrict` is pushed into the database query, so it filters rather than rejects and cannot be bypassed by a crafted `$filter`. This is why declarative restriction is required over handler checks.
- Grants MUST be specific. `grant: '*'` is acceptable only for a genuine administrator scope, and MUST be the last entry.
- Actions MUST be granted explicitly by name. An action is not covered by a `CREATE` or `UPDATE` grant.
- Where a rule cannot be expressed in `@restrict` — for example comparing order value against the user's `OrderValueLimit` attribute — implement it in a before handler on the action, and comment the reason.

Imperative check for a rule outside @restrict's reach

```

// Rule that @restrict cannot express: compare a data value to a user attribute.
async function confirmOrder(req) {
  const order = await SELECT.one.from(SalesOrders)
    .columns('ID', 'grossAmount', 'currency_code')
    .where({ ID: req.params[0].ID });

  const limit = Number(req.user.attr?.OrderValueLimit ?? 0);

  if (limit > 0 && Number(order.grossAmount) > limit) {
    return req.reject({
      code: 'ORDER_ABOVE_USER_LIMIT',
      message: { id: 'order.aboveUserLimit', values: [String(limit)] },
      status: 403
    });
  }

  // ... proceed with confirmation
}

```

9.4 Secrets and credentials

- Credentials MUST be obtained from Cloud Foundry service bindings (`VCAP_SERVICES`, surfaced by CAP as `cds.env.requires.<service>.credentials`) or from a BTP destination. Nothing else is acceptable.
- `.env`, `default-env.json`, and any file produced by `cf env` or `cds bind` MUST be listed in `.gitignore` and MUST NOT be committed. A secret that reaches Git history is compromised and MUST be rotated, not just deleted.
- The CI pipeline MUST run a secret scanner (Gitleaks or equivalent) on every pull request, and the check MUST be a required status.
- Deployment credentials MUST be held in the pipeline's secret store (Azure DevOps variable groups linked to Key Vault, or GitHub encrypted secrets / OIDC) and MUST NOT be echoed, even masked, in pipeline logs.
- Technical communication users MUST have least-privilege authorisations in S/4HANA, MUST be distinct per landscape, and MUST be rotated on the schedule the security policy defines.
- If a credential is ever pasted into a chat, ticket, or commit message, treat it as compromised and rotate it — do not simply delete the message.

9.5 Other controls

- All inbound traffic MUST be HTTPS. Applications MUST NOT expose an HTTP route.
- CAP's built-in security headers MUST remain enabled. Any relaxation of the content security policy requires an ADR.
- CSRF protection MUST remain enabled for browser-facing endpoints. Disabling it "because the tests are easier" is prohibited.
- `npm audit --audit-level=high` MUST pass. A known high or critical vulnerability with no available fix requires a time-boxed, ADR-recorded waiver with a compensating control.
- Any feature touching personal data (customer, practitioner, employee, or consumer records) MUST be raised with the Blackmores Data Privacy Officer before design is finalised, and the data flow recorded in the programme's processing register.

10. Integration Standards

10.1 S/4HANA consumption

- S/4HANA MUST be reached through a **BTP destination**, resolved at runtime. Hostnames, ports, clients, and credentials MUST NOT appear in code, in `mta.yaml`, or in `.cdsrc.json`.
- Only **released** S/4HANA APIs (OData or SOAP, published on SAP Business Accelerator Hub) may be consumed. Unreleased or custom-exposed CDS views require an ADR and an owner on the S/4HANA side committing to compatibility.
- The external service definition MUST be imported with `cds import` and the generated `.csn` committed. Hand-editing generated `.edmx` or `.csn` is prohibited — re-import instead, and record the API version in the commit message.
- Every external call MUST have an explicit timeout, a bounded retry policy with jitter, and a circuit breaker. A CAP service with no timeout on an S/4HANA call will hold its event loop and connection pool hostage during an S/4HANA incident.
- Retries MUST only be applied to idempotent operations. Retrying a non-idempotent POST creates duplicate business documents — a data-integrity defect, not a performance issue.

Importing an S/4HANA API definition

```
cds import ./API_SALES_ORDER_SRV.edmx --as cds --into srv/external
```

srv/lib/s4-order-client.js — timeout, bounded retry with jitter, safe error mapping

```
const cds = require('@sap/cds');
const log = cds.log('s4-order');

const TIMEOUT_MS    = Number(process.env.S4_REQUEST_TIMEOUT_MS ?? 15000);
const MAX_ATTEMPTS  = Number(process.env.S4_MAX_RETRY_ATTEMPTS ?? 3);

/**
 * Read a sales order from S/4HANA with bounded retry and timeout.
 * Idempotent (GET) – retry is safe.
 */
async function readS4Order(srv, orderNumber, correlationId) {
  const { A_SalesOrder } = srv.entities;

  for (let attempt = 1; attempt <= MAX_ATTEMPTS; attempt++) {
    try {
      return await srv.tx({ headers: { 'x-correlation-id': correlationId } }).run(
        SELECT.one.from(A_SalesOrder)
          .columns('SalesOrder', 'SalesOrderType', 'TotalNetAmount', 'TransactionCurrency')
          .where({ SalesOrder: orderNumber }),
        { timeout: TIMEOUT_MS }
      );
    } catch (error) {
      if (attempt === MAX_ATTEMPTS) {
        log.error('S/4HANA read failed after %d attempts: %s', MAX_ATTEMPTS, error);
      }
    }
  }
}
```

```

    );
  } catch (err) {
    const retryable = err.status >= 500 || err.code === 'ETIMEDOUT' || err.code === 'ECONNRESET';
    log.warn({ attempt, orderNumber, correlationId, status: err.status, retryable },
      'S/4HANA read failed');

    if (!retryable || attempt === MAX_ATTEMPTS) {
      throw Object.assign(new Error('error.s4Unavailable'),
        { code: 'S4_UNAVAILABLE', status: 502 });
    }

    // Exponential backoff with jitter
    await new Promise(r => setTimeout(r, (2 ** attempt) * 250 + Math.random() * 250));
  }
}

module.exports = { readS4Order };

```

ANTI-PATTERN Calling S/4HANA synchronously inside a user-facing read handler to enrich every row. One list request becomes fifty remote calls; response time becomes a function of S/4HANA load; and an S/4HANA outage takes the BTP application down with it. Replicate, cache, or call asynchronously instead.

10.2 Destination naming and configuration

Destination	Purpose	Authentication (PRD)
S4_API_SALES_ORDER_SRV	Sales order read/write	OAuth2SAMLBearerAssertion (principal propagation)
S4_API_BUSINESS_PARTNER	Customer master read	OAuth2SAMLBearerAssertion
S4_TECH_BATCH	Background/batch jobs with no user context	OAuth2ClientCredentials, technical user
EXT_<PARTNER>_<SERVICE>	Third-party API	As agreed per partner, documented in an ADR

- User-initiated calls MUST use **principal propagation** so S/4HANA authorisation applies to the actual user. A technical user for interactive scenarios collapses the audit trail and over-privileges every request.
- Technical users are permitted only for background jobs and event processing where no user context exists, and MUST use a destination whose name makes that explicit (`_TECH_`).
- Destination names MUST be identical across DEV, QAS, and PRD; only their targets differ. This is what allows the same MTA archive to be promoted unchanged.

10.3 Asynchronous messaging

- Use SAP Event Mesh for domain events where the producer does not need the consumer's answer. Do not use messaging to simulate request/response.
- Topic names MUST follow `blkm/<context>/<entity>/<event>/v1`, all lower case. The version segment is mandatory — payload evolution is inevitable.
- Event payloads MUST be small and MUST carry identifiers plus the minimum business context. Never publish an entire entity; consumers should read back what they need through the API.
- Consumers MUST be **idempotent**. At-least-once delivery means every consumer will eventually see a duplicate. Deduplicate on a natural key or a message ID, and record processed IDs.
- Publication MUST go through the transactional outbox (`cds.outboxed`) so an event is never emitted for a transaction that rolled back — and never lost for one that committed.
- Every consumer MUST have a dead-letter strategy and an alert on it. A silently failing consumer is indistinguishable from an idle one.

srv/handlers/stock-reserved-consumer.js — idempotent consumption

```
// Consumer: idempotent, with explicit dedup and structured logging.
const messaging = await cds.connect.to('messaging');

messaging.on('blkm/inventory/stock/reserved/v1', async (msg) => {
  const { messageId, data } = msg;

  const seen = await SELECT.one.from(ProcessedMessages).where({ messageId });
  if (seen) {
    log.info({ messageId }, 'duplicate message ignored');
    return;
  }

  await cds.tx(async (tx) => {
    await applyStockReservation(tx, data);
    await tx.run(INSERT.into(ProcessedMessages)
      .entries({ messageId, processedAt: new Date() }));
  });
});
```

11. SAP HANA Cloud Persistence Standards

11.1 Artefact generation

- HDI artefacts MUST be generated by `cds build --production`. Generated output under `gen/` MUST NOT be committed, MUST NOT be hand-edited, and MUST be in `.gitignore`.
- The `db` module MUST be deployed by the HDI deployer as part of the MTA. Deploying schema changes with `hdi-deploy` run manually from a laptop is prohibited in QAS and PRD.
- One HDI container per application per space. Sharing a container between applications couples their deployment lifecycles and removes the ability to release independently.
- Cross-container access, where genuinely required, MUST use synonyms configured through `.hdbgrants` and `.hdbsynonym`, be documented in an ADR, and be read-only.

11.2 Reference data via CSV

db/data/com.blackmores.order-OrderStatuses.csv

```
code;name;descr;criticality
DRAFT;Draft;Order is being prepared and not yet submitted;0
SUBMITTED;Submitted;Order submitted, awaiting confirmation;2
CONFIRMED;Confirmed;Order confirmed in S/4HANA;3
CANCELLED;Cancelled;Order cancelled;1
```

- The file name MUST be `<fully.qualified.namespace>-<EntityName>.csv`. CAP resolves the target entity from the file name; a mismatch means the file is silently ignored.
- CSVs are for **configuration and reference data only** — code lists, units, mapping tables. Transactional or master data MUST NOT be loaded this way.
- CSV content MUST be idempotent and complete for the entity, because HDI table-data loading replaces the managed rows. A partial CSV deletes data.
- CSVs MUST NOT contain personal data, even for testing. Test data belongs in `test/fixtures/`, which is never deployed.
- A CSV change is a data change and MUST be reviewed as one, including its effect on existing transactional rows that reference the codes being changed or removed.

11.3 Modelling for HANA performance

- Declare indexes for elements used in selective where clauses or joins that are not already part of the key. In CDS this is done with `@sql.append` on the entity, or a native `.hdbindex` under `db/src/` where the annotation is insufficient — either way it MUST be deliberate and documented.
- String lengths MUST be realistic. `String(5000)` used for a name wastes memory in HANA's column store and, worse, tells the next developer nothing about the real constraint.
- Monetary and quantity values MUST be `Decimal` with explicit precision and scale. `Double` for money is prohibited — floating-point rounding in financial figures is a defect that surfaces in reconciliation, months later.
- Avoid `LargeString` and `LargeBinary` in entities read as part of a collection; put them on a separate child entity so list reads do not drag the payload.
- Any query pattern expected to exceed 100 ms MUST be reviewed with an explain plan before merge, and the plan attached to the pull request.

```

entity SalesOrders : cuid, managed {
  orderNumber : String(10) @assert.unique;

  salesOrg    : Association to one SalesOrganisations;

  orderDate   : Date;

  // ...
}

// Deliberate secondary index for the most common access path
annotate SalesOrders with @sql.append: `
  , INDEX IDX_SALESORDERS_ORG_DATE (SALESORG_CODE, ORDERDATE DESC)
`;

```

11.4 Schema evolution

HDI deployment is incremental and will refuse changes it considers destructive. The rules below prevent a deployment failure from becoming an outage.

Change	HDI behaviour	Required approach
Add an optional element	Applied online	Normal release
Add a @mandatory element to a populated table	Fails — no default for existing rows	Add as optional, backfill, then add the constraint in a later release
Widen a String length	Applied online	Normal release
Narrow a length or change a type	Fails, or requires data loss	New element, migrate data, deprecate the old one over two releases
Rename an element or entity	Treated as drop + create — data loss	Add new, copy, deprecate. Never a direct rename on a populated table
Drop an element or entity	Blocked unless listed in <code>undeploy.json</code>	Explicit reviewed entry in <code>undeploy.json</code> , with a verified backup
Change a key	Not supported	New entity plus a migration plan; requires an ADR and change-board approval

RULE Any pull request that changes `db/` MUST state the HDI impact in its description, and any pull request touching `undeploy.json` MUST be approved by the technical lead and be accompanied by evidence of a current backup. Every schema change MUST be rehearsed in QAS against production-like volumes before PRD promotion.

12. Configuration, Profiles and Secrets

- Configuration MUST be supplied by CDS profiles (development, hybrid, production) and MTA extension descriptors — never by `if (process.env.NODE_ENV === ...)` branches scattered through the code.
- A configuration difference between landscapes MUST be expressed in the `.mtaext` for that landscape, so `mta.yaml` stays landscape-neutral and one archive can be promoted through all three.
- Defaults MUST be safe: the most restrictive setting is the default, and a landscape opts into anything looser. A missing variable must never silently enable something.
- The application MUST fail fast at startup if a required configuration value is absent, with a clear message naming the variable. Discovering a missing destination on the first user request is far more expensive than discovering it at boot.

mta.yaml — landscape-neutral deployment descriptor

```
_schema-version: '3.3'

ID: com.blackmores.btp.order

version: 1.5.0

parameters:

  enable-parallel-deployments: true

  keep-existing-routes: true

build-parameters:

  before-all:

    - builder: custom

      commands:

        - npm ci

        - npx cds build --production

modules:

  - name: blkm-btp-order-srv

    type: nodejs

    path: gen/srv

    parameters:

      buildpack: nodejs_buildpack

      memory: 512M

      disk-quota: 1024M

      instances: 2

      health-check-type: http

      health-check-http-endpoint: /health
```

build-parameters:

builder: npm-ci

requires:

- name: blkm-btp-order-db
- name: blkm-btp-order-uaa
- name: blkm-btp-order-dest
- name: blkm-btp-order-logs

provides:

- name: srv-api

properties:

srv-url: \${default-url}

- name: blkm-btp-order-db-deployer

type: hdb

path: gen/db

parameters:

buildpack: nodejs_buildpack

memory: 256M

requires:

- name: blkm-btp-order-db

resources:

- name: blkm-btp-order-db

type: com.sap.xs.hdi-container

parameters:

service: hana

service-plan: hdi-shared

- name: blkm-btp-order-uaa

type: org.cloudfoundry.managed-service

parameters:

service: xsuaa

service-plan: application

path: ./xs-security.json

config:

xsappname: blkm-btp-order-\${space}

```

    tenant-mode: dedicated

- name: blkm-btp-order-dest
  type: org.cloudfoundry.managed-service
  parameters:
    service: destination
    service-plan: lite

- name: blkm-btp-order-logs
  type: org.cloudfoundry.managed-service
  parameters:
    service: application-logs
    service-plan: lite

```

prd.mtaext — the only place landscape differences live

```

_schema-version: '3.3'
ID: com.blackmores.btp.order.prd
extends: com.blackmores.btp.order

modules:
- name: blkm-btp-order-srv
  parameters:
    instances: 4
    memory: 1024M
  properties:
    CDS_ENV: production
    S4_REQUEST_TIMEOUT_MS: '20000'
    S4_MAX_RETRY_ATTEMPTS: '3'
    FEATURE_ORDER_SPLIT_ENABLED: 'false'

resources:
- name: blkm-btp-order-db
  parameters:
    service-plan: hdi-shared
- name: blkm-btp-order-logs
  parameters:
    service: cloud-logging

```

service-plan: standard

13. Logging and Observability

13.1 Logging rules

- `console.log`, `console.error`, and `process.stdout.write` are prohibited. Use `cds.log(<component>)`, which emits structured JSON that the Cloud Logging / Application Logging service can index.
- Create one logger per module with a stable component name: `cds.log('sales-order')`. The component name is what an on-call engineer filters on.
- Logs **MUST** be structured: pass an object of fields first, then a short message. Interpolating values into a message string makes them unsearchable.
- Every log line for a request **MUST** carry the correlation ID (`x-correlation-id`, propagated by CAP). Without it, a single user complaint cannot be traced across the service, HANA, and S/4HANA.
- Personal data, credentials, tokens, and full request bodies **MUST NOT** be logged. Log identifiers, not payloads. Where a payload is needed for diagnosis, log a hash or a redacted subset.
- Log levels: `error` for a failure needing intervention, `warn` for a handled anomaly worth watching, `info` for business-significant milestones, `debug` for development. `debug` **MUST** be off in PRD.

Structured logging

```
const cds = require('@sap/cds');
const log = cds.log('sales-order');

// CORRECT – structured, searchable, no personal data
log.info({
  orderId:      order.ID,
  orderNumber:  order.orderNumber,
  salesOrg:     salesOrg,
  correlationId: req.headers['x-correlation-id']
}, 'order confirmed');

// PROHIBITED – unsearchable, and leaks personal data into a log store
console.log('Order ' + order.orderNumber + ' confirmed for ' +
  customer.name + ' <' + customer.email + '>');
```

13.2 Health, metrics and tracing

- Every service **MUST** expose a `/health` endpoint returning 200 only when the application can serve traffic, including a shallow HANA connectivity check. The MTA health check **MUST** point at it.
- A `/readiness` check **SHOULD** additionally verify destination resolvability, so a misconfigured destination surfaces at deploy time rather than on first use.
- Service-level indicators **MUST** be defined before go-live: p95 response time per endpoint, error rate, S/4HANA call failure rate, and event-consumer lag. Alert thresholds **MUST** be agreed with the support team and documented in the runbook.
- Distributed tracing **SHOULD** be enabled via OpenTelemetry (`@cap-js/telemetry`) so a request can be followed from the UI through CAP to HANA and S/4HANA.

```
const cds = require('@sap/cds');

cds.on('bootstrap', (app) => {
  app.get('/health', async (_req, res) => {
    try {
      await cds.db.run('SELECT 1 FROM DUMMY');
      res.status(200).json({ status: 'UP', version: process.env.npm_package_version });
    } catch (err) {
      cds.log('health').error({ err: err.code }, 'database probe failed');
      res.status(503).json({ status: 'DOWN' });
    }
  });
});

module.exports = cds.server;
```

14. Performance and Resilience

Concern	Standard
Payload size	Explicit column lists and an enforced <code>@cds.query.limit</code> on every collection. No <code>SELECT *</code> in application code.
Round trips	No database or remote reads inside loops. Batch by key set; use <code>\$batch</code> or a single <code>IN</code> predicate.
Deep reads	Expand only what the consumer needs. An unbounded <code>\$expand</code> across three levels of composition can return tens of thousands of rows from one request.
Remote calls	Mandatory timeout, bounded retry with jitter, circuit breaker. Retry only idempotent operations.
Caching	Cache slow-changing reference data in memory with an explicit TTL and an invalidation path. Never cache user-specific or authorisation-dependent data in module scope.
Long-running work	Offload to an Event Mesh consumer or a scheduled job. No request handler may hold a transaction across a remote call chain.
Scaling	Minimum two instances in QAS and PRD. Services MUST be stateless so instances are interchangeable and horizontal scaling works.
Connection pool	HANA pool sizing reviewed against instance count so instances × pool size stays within the HANA Cloud connection limit.
Graceful shutdown	Handle <code>SIGTERM</code> : stop accepting new work, let in-flight requests finish, close the pool. Cloud Foundry restages and rebalances routinely.
Load testing	Every release touching a high-volume path MUST be load tested in QAS against production-like volumes before PRD promotion.

ANTI-PATTERN Storing per-user or per-request state in a module-level variable "for performance". Cloud Foundry runs multiple instances behind a load balancer, so the cache hit rate is unpredictable and, worse, one user can be served another user's data. This is a security defect, not an optimisation.

15. Testing Standards

15.1 Test pyramid and coverage

Level	Scope	Runtime	Coverage requirement
Unit	Pure functions in <code>srv/lib/</code> : calculations, mappers, validators	No database, no HTTP, no CAP runtime	At least 90% of <code>srv/lib/</code>
Service / integration	CAP service through <code>cds.test</code> : OData requests, handlers, authorisation, <code>@restrict</code>	In-memory SQLite, mocked auth, mocked externals	At least 80% statements and branches overall
Contract	Imported external service definitions against recorded S/4HANA responses	Mock server from committed fixtures	Every consumed API operation
End-to-end	Deployed QAS landscape with real destinations	QAS	Agreed critical business paths only

- Overall line and branch coverage **MUST** be at least **80%**, and the pipeline **MUST** fail below it. The threshold is a floor, not a target.
- Every authorisation rule **MUST** have a negative test proving the unauthorised case is rejected. An authorisation rule with only a positive test is untested.
- Every bug fix **MUST** add a test that fails before the fix and passes after. This is the only defence against regression of the same defect.
- Tests **MUST NOT** depend on execution order, on wall-clock time, or on data left behind by another test. Use `beforeEach` to reset state.
- A test that is skipped (`it.skip`, `xit`) **MUST** carry a ticket reference in a comment or be deleted. Permanently skipped tests give false assurance.

15.2 Service tests with `cds.test`

test/integration/sales-order-service.integration.test.js

```
const cds = require('@sap/cds');

const { GET, POST, expect, axios } = cds.test(__dirname + '/../..');
axios.defaults.validateStatus = () => true; // assert on status explicitly

const PROCESSOR = { username: 'order.processor', password: 'initial' };
const VIEWER = { username: 'order.viewer', password: 'initial' };

describe('SalesOrderService', () => {

  describe('authorisation', () => {
```

```

it('rejects anonymous reads with 401', async () => {
  const res = await GET('/api/v1/sales-order/SalesOrders');
  expect(res.status).toEqual(401);
});

it('denies create to a read-only role', async () => {
  const res = await POST('/api/v1/sales-order/SalesOrders',
    { orderDate: '2026-09-15', salesOrg_code: '1710' }, { auth: VIEWER });
  expect(res.status).toEqual(403);
});

it('filters reads to the permitted sales organisations', async () => {
  const res = await GET('/api/v1/sales-order/SalesOrders?$select=salesOrg_code',
    { auth: PROCESSOR });
  expect(res.status).toEqual(200);
  expect(res.data.value.every(o => o.salesOrg_code === '1710')).toBe.true;
});
});

describe('validation', () => {

  it('rejects a non-positive quantity with a targeted error', async () => {
    const res = await POST('/api/v1/sales-order/SalesOrderItems',
      { quantity: 0, product_ID: PRODUCT_ID }, { auth: PROCESSOR });
    expect(res.status).toEqual(400);
    expect(res.data.error.code).toEqual('ITEM_QTY_INVALID');
    expect(res.data.error.target).toEqual('quantity');
  });
});

describe('confirmOrder', () => {

  it('rejects confirmation above the user value limit', async () => {
    const url = '/api/v1/sales-order/SalesOrders(' + HIGH_VALUE_ORDER_ID +
      ')/SalesOrderService.confirmOrder';
  });
});

```

```

const res = await POST(url, { confirmationNote: 'ok' }, { auth: PROCESSOR });

expect(res.status).toEqual(403);

expect(res.data.error.code).toEqual('ORDER_ABOVE_USER_LIMIT');

});

});

});

```

15.3 Test data and external mocks

- Test data MUST live in `test/fixtures/` and MUST NOT be placed in `db/data/`, or it will be deployed to a real landscape.
- Test data MUST be synthetic. Production extracts — even masked — MUST NOT be committed to Git. Any requirement to test against real data volumes is met in QAS with a controlled, approved masked load, and must be cleared with the Data Privacy Officer.
- External services MUST be mocked in integration tests, using either `cds mock` with the imported service definition or a recorded-fixture mock server. Tests MUST NOT call a real S/4HANA system: it makes the suite slow, flaky, and dependent on someone else's data.
- Contract fixtures MUST be refreshed whenever the S/4HANA API version changes, and the refresh MUST be a visible, reviewed commit.

15.4 Naming tests

- A `describe` block names the unit under test; a nested `describe` names the behaviour area; the `it` block completes the sentence "it ...".
- `it('rejects confirmation above the user value limit')` is acceptable. `it('test1')`, `it('works')`, and `it('should work correctly')` are not.
- One assertion concern per test. A test asserting six unrelated things reports one failure and hides the other five.

16. Code Quality, Linting and Formatting

- `cds-lint` and ESLint MUST both pass with zero warnings (`--max-warnings 0`). A warning that is acceptable MUST be disabled explicitly with a rule name and a reason comment, never by a blanket file-level disable.
- Prettier is the single formatting authority. Formatting MUST NOT be discussed in code review; the pipeline checks it.
- Husky hooks MUST be installed on clone (via the `prepare` script) so the checks run before a commit exists.
- Dead code, commented-out blocks, and unreferenced files MUST be deleted. Git is the history.
- TODO comments MUST include a ticket reference: `// TODO(BLK-1234): replace with Event Mesh consumer`. A TODO with no ticket is a permanent apology.
- Doc comments are mandatory on every exported function and every CDS entity, service, action, and event.

.eslintrc.json

```
{
  "root": true,
  "extends": ["eslint:recommended", "plugin:@sap/cds/recommended"],
  "parserOptions": { "ecmaVersion": 2023, "sourceType": "script" },
  "env": { "node": true, "es2023": true, "jest": true },
  "rules": {
    "no-console": "error",
    "no-unused-vars": ["error", { "argsIgnorePattern": "^_" }],
    "no-await-in-loop": "error",
    "require-await": "error",
    "no-return-await": "error",
    "eqeqeq": ["error", "always"],
    "prefer-const": "error",
    "no-var": "error",
    "complexity": ["error", 12],
    "max-lines-per-function": ["warn", 50],
    "max-depth": ["error", 3],
    "camelcase": ["error", { "properties": "never", "allow": ["^[a-z]+_[a-z]+$"] }]
  }
}
```

`no-await-in-loop` is set to error deliberately: it is the lint rule that catches the N+1 anti-pattern before it reaches review. Where sequential awaiting is genuinely required — for example rate-limited external calls — disable it on the specific line with a reason. The `camelcase` allowance exists because CAP generates foreign-key elements such as `salesOrg_code`.

commitlint.config.js

```
module.exports = {
```

```
extends: ['@commitlint/config-conventional'],
rules: {
  'scope-enum': [2, 'always',
    ['db', 'srv', 'security', 'deploy', 'order', 'product', 'customer', 'test', 'docs', 'ci']],
  'subject-case': [2, 'always', 'sentence-case'],
  'header-max-length': [2, 'always', 72],
  'references-empty': [2, 'never']
},
parserPreset: { parserOpts: { issuePrefixes: ['BLK-'] } }
};
```

17. Git Workflow and Versioning

17.1 Branching model

The programme uses a GitFlow-lite model with two long-lived branches, matched to the three landscapes.

Branching model

```
main      --o-----o-----o----->  deploys to QAS, tagged for PRD
          \           /           /
develop   ---o---o---o---o---o-----o---o---o---o----->  deploys to DEV
          \           /           \           /
          feature/BLK-1234   feature/BLK-1290

hotfix/BLK-1402 -----o---->  branched from the PRD tag,
                               merged to BOTH main and develop
```

- `main` and `develop` MUST be protected: no direct pushes, no force pushes, linear history, required status checks, and at least one approving review.
- A feature branch MUST be short-lived — three working days is the target, five the limit. Long-lived branches produce large, unreviewable merges.
- Feature branches MUST be kept current by **rebasing** onto `develop`, and merged with squash so `develop` carries one commit per work item.
- A hotfix MUST be branched from the PRD release tag, not from `develop`, and MUST be merged into both `main` and `develop`. Forgetting the second merge means the fix regresses at the next release — the most common cause of "we fixed that already".
- Release notes MUST be generated from Conventional Commit messages, which is the practical reason the commit format is enforced.

17.2 Pull requests

- A pull request SHOULD change fewer than 400 lines. Beyond that, review quality falls sharply and defects pass through.
- The description MUST state: what changed, why, the work item, the HDI/schema impact, the authorisation impact, and how it was tested.
- Reviewers MUST use the checklist in Appendix B. Approval without engaging with the checklist is not a review.
- A pull request MUST NOT mix a refactor with a behaviour change. Reviewers cannot see the behaviour change inside a large diff of moved code.

18. CI/CD — Azure DevOps

The pipeline below is the reference implementation. Its stages and gate order are mandatory; the tooling specifics may be adapted to the organisation's agent pool.

.pipelines/azure-pipelines.yml

```
trigger:
  branches: { include: [ develop, main, 'release/*' ] }

pr:
  branches: { include: [ develop, main ] }

variables:
  - group: blkm-btp-common
  - name: NODE_VERSION
    value: '22.x'
  - name: MTAR
    value: 'blkm-btp-order.mtar'

stages:

# ----- 1. Build & verify -----
- stage: Verify
  jobs:
  - job: BuildTest
    pool: { vmImage: 'ubuntu-latest' }
    steps:
      - task: NodeTool@0
        inputs: { versionSpec: $(NODE_VERSION) }

      - script: npm ci
        displayName: 'Install (locked)'

      - script: npm run lint
        displayName: 'Lint (cds-lint + eslint, zero warnings)'

      - script: npm run format:check
        displayName: 'Prettier check'
```

```

- script: npx commitlint --from=origin/${System.PullRequest.TargetBranch} --to=HEAD
  displayName: 'Commit message check'
  condition: eq(variables['Build.Reason'], 'PullRequest')

- script: npm test
  displayName: 'Unit + integration tests with coverage'

- task: PublishTestResults@2
  inputs: { testResultsFiles: '**/junit*.xml', mergeTestResults: true }
  condition: always()

- task: PublishCodeCoverageResults@2
  inputs: { summaryFileLocation: 'coverage/cobertura-coverage.xml' }

- script: npm audit --audit-level=high
  displayName: 'Dependency vulnerability scan'

- script: |
  curl -sSfL https://raw.githubusercontent.com/gitleaks/gitleaks/master/scripts/install.sh | sh
  ./gitleaks detect --no-git --redact --exit-code 1
  displayName: 'Secret scan'

- task: SonarQubeAnalyze@5
  displayName: 'Static analysis'

- task: SonarQubePublish@5
  inputs: { pollingTimeoutSec: '300' }
  displayName: 'Quality gate (blocking)'

- script: |
  npm i -g mbt
  mbt build -p=cf --mtar=$(MTAR)
  displayName: 'MTA build'

- publish: 'mta_archives/$(MTAR)'
  artifact: mtar
  displayName: 'Publish the single promotable artefact'

```

```

# ----- 2. DEV -----
- stage: DeployDev
  dependsOn: Verify
  condition: and(succeeded(), eq(variables['Build.SourceBranch'], 'refs/heads/develop'))
  jobs:
    - deployment: Dev
      environment: 'blkm-btp-dev'
      strategy:
        runOnce:
          deploy:
            steps:
              - download: current
                artifact: mtar
              - task: CloudFoundryCLIInstall@0
              - script: |
                  cf api $(CF_API_DEV)
                  cf auth "$(CF_USER)" "$(CF_PASSWORD)"
                  cf target -o $(CF_ORG_DEV) -s dev
                  cf deploy $(Pipeline.Workspace)/mtar/$(MTAR) \
                      -e dev.mtaext -f --delete-services=false
                  displayName: 'cf deploy (DEV)'

# ----- 3. QAS -----
- stage: DeployQas
  dependsOn: Verify
  condition: and(succeeded(), eq(variables['Build.SourceBranch'], 'refs/heads/main'))
  jobs:
    - deployment: Qas
      environment: 'blkm-btp-qas'
      strategy:
        runOnce:
          deploy:
            steps:
              - download: current
                artifact: mtar

```

```

- task: CloudFoundryCLIInstall@0

- script: |

    cf api $(CF_API_QAS)

    cf auth "$(CF_USER)" "$(CF_PASSWORD)"

    cf target -o $(CF_ORG_QAS) -s qas

    cf deploy $(Pipeline.Workspace)/mtar/$(MTAR) -e qas.mtaext -f

    displayName: 'cf deploy (QAS)'

- script: npm run test:e2e

    displayName: 'Smoke / E2E against QAS'

# ----- 4. PRD (manual approval on the environment) -----

- stage: DeployPrd

    dependsOn: DeployQas

    condition: and(succeeded(), startsWith(variables['Build.SourceBranch'], 'refs/tags/v'))

    jobs:

    - deployment: Prd

        environment: 'blkm-btp-prd'      # approval + change-record check configured here

        strategy:

            runOnce:

                deploy:

                    steps:

                    - download: current

                        artifact: mtar      # SAME archive tested in QAS - never rebuilt

                    - task: CloudFoundryCLIInstall@0

                    - script: |

                        cf api $(CF_API_PRD)

                        cf auth "$(CF_USER)" "$(CF_PASSWORD)"

                        cf target -o $(CF_ORG_PRD) -s prd

                        cf deploy $(Pipeline.Workspace)/mtar/$(MTAR) \

                            -e prd.mtaext -f --strategy blue-green --skip-testing-phase=false

                        displayName: 'Blue-green deploy (PRD)'

```

RULE The artefact built in the Verify stage is the only artefact deployed to any landscape. Rebuilding per environment is prohibited: it means PRD runs a binary that was never tested, which defeats the entire purpose of the pipeline.

19. CI/CD — GitHub Actions

Where a repository is hosted on GitHub, the equivalent workflow below applies. The gates are identical; only the syntax differs.

.github/workflows/ci.yml

```
name: CI

on:
  pull_request:
    branches: [ develop, main ]
  push:
    branches: [ develop, main ]
    tags: [ 'v*' ]

permissions:
  contents: read
  id-token: write      # OIDC - no long-lived deployment credentials in secrets

jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }
      - uses: actions/setup-node@v4
        with: { node-version: '22', cache: 'npm' }

      - run: npm ci
      - run: npm run lint
      - run: npm run format:check

      - name: Commit message check
        if: github.event_name == 'pull_request'
        run: npx commitlint --from=${{ github.event.pull_request.base.sha }} --to=HEAD

      - run: npm test

      - name: Enforce coverage floor
```

```

    run: |
        npx jest --coverage --coverageThreshold='{ "global": { "statements": 80,
        "branches": 80, "functions": 80, "lines": 80 } }'

- run: npm audit --audit-level=high
- uses: gitleaks/gitleaks-action@v2

- name: MTA build
  run: |
    npm i -g mbt
    mbt build -p=cf --mtar=blkm-btp-order.mtar

- uses: actions/upload-artifact@v4
  with:
    name: mtar
    path: mta_archives/blkm-btp-order.mtar
    retention-days: 30

deploy-dev:
  needs: verify
  if: github.ref == 'refs/heads/develop'
  runs-on: ubuntu-latest
  environment: blkm-btp-dev
  steps:
    - uses: actions/download-artifact@v4
      with: { name: mtar }
    - name: Deploy to DEV
      run: |
        cf api "${{ vars.CF_API_DEV }}"
        cf auth "${{ secrets.CF_USER }}" "${{ secrets.CF_PASSWORD }}"
        cf target -o "${{ vars.CF_ORG_DEV }}" -s dev
        cf deploy blkm-btp-order.mtar -e dev.mtaext -f

deploy-prd:
  needs: verify
  if: startsWith(github.ref, 'refs/tags/v')

```

```

runs-on: ubuntu-latest

environment: blkm-btp-prd          # required reviewers configured on the environment

steps:
  - uses: actions/download-artifact@v4
    with: { name: mtar }
  - name: Blue-green deploy to PRD
    run: |
      cf api "${{ vars.CF_API_PRD }}"
      cf auth "${{ secrets.CF_USER }}" "${{ secrets.CF_PASSWORD }}"
      cf target -o "${{ vars.CF_ORG_PRD }}" -s prd
      cf deploy blkm-btp-order.mtar -e prd.mtaext -f --strategy blue-green

```

- GitHub environment protection rules MUST supply the PRD approval gate; approval MUST NOT be implemented as a manual `workflow_dispatch` that any contributor can trigger.
- Deployment identities SHOULD use OIDC federation rather than stored passwords wherever the platform supports it.
- Workflow files MUST pin third-party actions to a full commit SHA or a maintained major tag. An unpinned action is a supply-chain risk in the deployment path.

20. Deployment and Promotion

20.1 Promotion rules

1. The `.mtar` is built **once**, in the Verify stage, from a specific commit. Its SHA-256 is recorded in the build record.
2. That archive is deployed to DEV with `dev.mtaext`, then to QAS with `qas.mtaext`. Landscape differences live only in the extension descriptor.
3. PRD promotion requires: a green QAS run against the same archive, a signed annotated tag `v<semver>`, an approved change record, a reviewed schema-change plan if db/ changed, and a documented rollback plan.
4. PRD deployment uses `--strategy blue-green`, which stages the new version alongside the old and switches the route only after the deployed application reports healthy.
5. Post-deployment verification is mandatory and **MUST** be recorded: `/health` returns 200, a smoke test of the critical path passes, error rate and p95 latency are within threshold for 30 minutes.

20.2 Rollback

- Application rollback is a blue-green route switch back, or a re-deployment of the previous `.mtar`. Both **MUST** be rehearsed in QAS before the first PRD release.
- **Database changes are not rolled back by redeploying the application.** A schema change **MUST** therefore be designed to be backward compatible with the previous application version — add columns, never remove or narrow them in the same release. This is what makes application rollback safe.
- A destructive schema change **MUST** be split across at least two releases: release N adds the new structure and dual-writes; release N+1 removes the old one once N is proven. `undeploy.json` entries only ever appear in the second release.
- Every release **MUST** have a named rollback decision-maker and a maximum decision window agreed before deployment.

20.3 Release checklist

#	Check	Owner
1	Version bumped consistently in <code>package.json</code> and <code>mta.yaml</code> ; CI version-match check green	Developer
2	Release notes generated from Conventional Commits and reviewed	Technical lead
3	Schema changes reviewed; HDI impact documented; backward compatibility confirmed	Technical lead + DBA
4	New or changed XSUAA scopes documented with the role-collection action required per landscape	Technical lead + BTP admin
5	Destinations required by this release exist and are correctly configured in the target landscape	BTP admin

6	QAS pipeline green on the identical artefact; smoke and regression suites passed	QA
7	Load test evidence attached for any change on a high-volume path	QA
8	Rollback plan documented; decision-maker and decision window named	Release manager
9	Change record approved; deployment window confirmed with the business	Release manager
10	Runbook and monitoring thresholds updated for anything new	Technical lead
11	Post-deployment verification executed and evidence recorded	Release manager

21. Documentation and Definition of Done

21.1 Required documentation

- `README.md` MUST tell a new developer how to clone, install, run locally, run the tests, and deploy — and MUST be verified by someone who did not write it.
- Architecture decisions MUST be recorded as ADRs in `docs/decisions/`, numbered sequentially, with Context, Decision, Consequences, and Status. Every standard waiver is an ADR.
- Doc comments on CDS entities, services, actions, and events are mandatory; CAP surfaces them into the generated metadata, so they become the API documentation.
- A runbook MUST exist before the first PRD deployment, covering: architecture summary, dependencies, health checks, log queries for common symptoms, alert thresholds, known failure modes with remediation, escalation path, and rollback procedure.

21.2 Definition of Done

A work item is Done only when every one of the following is true. Partial completion is not Done, and is not reported as Done.

1. Acceptance criteria met and demonstrated to the product owner.
2. Code complies with this standard; lint, format, and commit checks pass with zero warnings.
3. Unit and integration tests written, including a negative test for every authorisation rule touched; coverage floor met.
4. A regression test exists for any defect fixed.
5. Authorisation impact assessed; `@restrict` and `xs-security.json` reviewed by the technical lead if changed.
6. Schema impact assessed; HDI deployment rehearsed in DEV, then QAS; backward compatibility confirmed.
7. Doc comments, README, and runbook updated where behaviour or operations changed.
8. An ADR recorded for any architectural decision or standard waiver.
9. Pull request approved by at least one other engineer using the Appendix B checklist; all CI gates green.
10. Deployed and verified in DEV, then QAS; no new errors or latency regression observed.
11. No `TODO` without a ticket, no skipped test without a ticket, no secret in the diff.

22. Anti-Pattern Catalogue

Each entry below has been observed to cause real defects in CAP projects. Presence of any of these in a pull request is grounds for rejection. Severity indicates the consequence if it reaches production.

22.1 Modelling anti-patterns

Anti-pattern	Why it is harmful	Do this instead	Severity
Service entity redefined instead of projected	The service model diverges silently from the domain model; a db change does not propagate and the two drift until a defect surfaces.	Always as projection on <code>db.<Entity></code> with an explicit excluding.	High
Composition used for a master-data reference	Deleting a transaction cascade-deletes shared master data.	Association for references; Composition only for owned children.	Critical
Composite / semantic primary keys	Propagates into every association, URL, and fixture; a business-key change becomes a schema migration.	<code>cuid</code> UUID key plus <code>@assert.unique</code> on the business key.	High
Double for money or quantity	Floating-point rounding produces reconciliation errors that surface months later in finance.	Decimal with explicit precision and scale.	Critical
Validation logic duplicating <code>@mandatory</code> / range checks	Two sources of truth, inconsistent error payloads, and the handler path is bypassed by draft activation and CSV load.	Declarative CDS annotations; handlers only for multi-field or stateful rules.	Medium
Descriptive text not declared localized	Retrofitting localisation later requires a data migration into the generated <code>.texts</code> entity.	Declare localized from the start; use <code>sap.common.CodeList</code> .	Medium
Entity or element renamed on a populated table	HDI treats a rename as drop + create — the data is lost.	Add new, migrate, deprecate over two releases.	Critical
<code>String(5000)</code> used everywhere "to be safe"	Wastes HANA column-store memory and destroys the documentary value of the model.	Realistic, business-derived lengths.	Low

22.2 Service and logic anti-patterns

Anti-pattern	Why it is harmful	Do this instead	Severity
String-concatenated SQL	SQL-injectable, HANA-dialect-locked, untestable on SQLite, invisible to CDS authorisation.	<code>cds.q1</code> with bound values, exclusively.	Critical
Database or remote read inside a loop (N+1)	Latency scales with row count; passes every test that uses three fixture rows.	Batch: one <code>SELECT ... where ID in [...]</code> ; <code>no-await-in-loop</code> catches it at lint time.	High
<code>SELECT *</code> with no <code>.limit()</code>	Unbounded payload and memory growth as data volume grows.	Explicit columns, explicit limit, <code>@cds.query.limit</code> on the entity.	High
<code>await super.init()</code> called first in <code>init()</code>	CAP generic handlers register ahead of yours, so <code>before</code> hooks run after persistence. Silent, and very hard to diagnose.	Always the last statement of <code>init()</code> .	High
All logic in <code>srv/<service>.js</code>	A 1,500-line file nobody can review; merge conflicts on every story.	Thin registration file; logic in <code>srv/handlers/</code> and pure code in <code>srv/lib/</code> .	Medium
Module-level mutable state used as a cache	Multiple CF instances and concurrent requests mean cross-request data leakage — a security defect.	Stateless services; request-scoped state; an explicit cache with TTL for non-user data only.	Critical
Writing to the database in an <code>after</code> handler	Outside the intended transaction semantics; failures surface after the client believes it succeeded.	<code>before</code> for mutation; <code>after</code> only for shaping the response.	High
<code>commit()</code> / <code>rollback()</code> called inside a handler	Breaks CAP's request transaction and leaves partial writes.	Throw or <code>req.error()</code> to roll back; return to <code>commit</code> .	High
<code>cds.connect.to()</code> inside a request handler	Re-resolves the destination on every request, adding latency and load on the destination service.	Connect once in <code>init()</code> ; hold on the service instance.	Medium
<code>console.log</code> for diagnostics	Unstructured, unindexed, and frequently leaks personal data into the log store.	<code>cds.log(<component>)</code> with structured fields and a correlation ID.	Medium

A function that writes data	OData functions are treated as safe and cacheable; clients and gateways will repeat them.	Any mutation is an action.	High
Floating promise (missing await)	Unhandled rejection that CAP cannot attribute to a request; work silently does not happen.	await every promise; enable require-await in lint.	High

22.3 Security anti-patterns

Anti-pattern	Why it is harmful	Do this instead	Severity
Service with no @requires	Open to every authenticated caller in the subaccount, including other applications.	Explicit @requires at service level, @restrict per entity.	Critical
@requires: 'any' left in after local testing	Unauthenticated public endpoint in production.	Mocked auth via the development profile; never weaken the model for testing.	Critical
Authorisation enforced only in a handler	Bypassed by any path the author did not anticipate; a crafted \$filter can leak rows.	@restrict with a where clause, pushed into the query.	Critical
One coarse scope for the whole application	No separation of duties; a viewer can confirm orders.	One scope per capability; role templates per job.	High
.env / default-env.json committed	Credentials in Git history are compromised permanently and must be rotated.	.gitignore plus a blocking secret scan in CI.	Critical
S/4HANA hostname or client hardcoded	Cannot promote the same artefact; a landscape mix-up can write test data into production.	BTP destinations, resolved at runtime.	High
Technical user for interactive scenarios	Collapses the audit trail and over-privileges every request.	Principal propagation for user-initiated calls.	High
Stack trace or SQL text returned to the client	Discloses internal structure to an attacker.	Central error handler that maps 5xx to a safe payload.	High
CSRF or security headers disabled	Reintroduces well-understood web vulnerabilities.	Leave CAP defaults on; any relaxation requires an ADR.	High

Personal data in logs or error messages	Creates an uncontrolled copy of personal data in a log store with different retention.	Log identifiers only; raise any personal-data flow with the Data Privacy Officer.	Critical
---	--	---	-----------------

22.4 Integration anti-patterns

Anti-pattern	Why it is harmful	Do this instead	Severity
S/4HANA call per row in a list read	One list request becomes N remote calls; response time tracks S/4HANA load; an S/4 outage takes BTP down.	Replicate or cache; or enrich asynchronously.	Critical
No timeout on an external call	A slow S/4HANA system exhausts the event loop and connection pool.	Explicit timeout, bounded retry with jitter, circuit breaker.	Critical
Retrying a non-idempotent POST	Creates duplicate business documents.	Retry only idempotent operations; use an idempotency key otherwise.	Critical
Hand-edited generated <code>.edmx</code> / <code>.csn</code>	Lost at the next re-import; the model no longer matches the real API.	Re-run <code>cds import</code> ; record the API version in the commit.	Medium
Messaging used for request/response	Couples producer to consumer availability while losing HTTP's error semantics.	Synchronous call for queries; events only for notification.	High
Non-idempotent event consumer	At-least-once delivery guarantees eventual duplicate processing.	Deduplicate on message ID; record processed IDs transactionally.	Critical
Event published outside the transaction	Event emitted for a rolled-back transaction, or lost for a committed one.	<code>cds.outboxed(messaging).emit(...)</code> .	High
Full entity published as the event payload	Leaks internal fields, couples consumers to the internal model, and bloats the topic.	Identifiers plus minimum context; consumers read back via the API.	Medium

22.5 Delivery and DevOps anti-patterns

Anti-pattern	Why it is harmful	Do this instead	Severity
--------------	-------------------	-----------------	----------

<code>cf push</code> instead of MTA deployment	Bypasses service binding and HDI deployment; the landscape drifts from the descriptor and cannot be reproduced.	<code>mbt build</code> then <code>cf deploy</code> of the <code>.mtar</code> .	Critical
Rebuilding the archive per environment	PRD runs a binary that was never tested.	Build once; promote the same <code>.mtar</code> with a landscape <code>.mtaext</code> .	Critical
Landscape encoded in resource names or <code>mta.yaml</code>	Forces landscape-specific descriptors and guarantees they diverge.	Neutral <code>mta.yaml</code> plus <code>dev/qas/prd</code> extension descriptors.	High
<code>npm install</code> in CI instead of <code>npm ci</code>	Resolves different dependency versions than were tested locally.	<code>npm ci</code> with a committed <code>package-lock.json</code> .	High
<code>gen/</code> or <code>node_modules/</code> committed	Enormous diffs, merge conflicts, and stale generated artefacts deployed.	<code>.gitignore</code> ; regenerate in the build.	Medium
Long-lived feature branches	Large unreviewable merges and repeated conflict resolution.	Short-lived branches, rebase onto <code>develop</code> , squash merge.	Medium
Hotfix merged to <code>main</code> only	The fix regresses at the next release from <code>develop</code> .	Merge every hotfix into both <code>main</code> and <code>develop</code> .	High
Skipping or lowering the coverage gate to ship	Removes the only automated check on regression risk.	Fix the tests; a threshold change needs an ADR.	High
Destructive schema change in the same release as the code that stops using it	Application rollback becomes impossible.	Two-release pattern: add and dual-write, then remove.	Critical
Manual change made directly in a landscape	The landscape no longer matches Git; the next deployment silently reverts it.	Every change through the pipeline; no exceptions in QAS or PRD.	Critical

Appendix A — Naming Quick Reference

A.1 CDS layer

Artefact	Pattern	Correct	Incorrect
Namespace	com.blackmores.<context>	com.blackmores.order	blackmores_order, com.BLMK.Order
Entity	PascalCase plural	SalesOrders	salesOrder, TBL_ORDERS, SalesOrderEntity
Element	camelCase singular	grossAmount	GrossAmount, gross_amount, amtGross
Boolean element	predicate form	isBlocked, hasAttachments	blocked, blockedFlag, FLAG_BLOCKED
Type	PascalCase singular	Amount	amountType, TAmount
Aspect	PascalCase adjectival	Auditable	AuditFields, BaseEntity
Enum value	UPPER_SNAKE with literal	IN_DELIVERY = 'IN_DELIVERY'	inDelivery, implicit values
To-one association	camelCase singular role	shipToParty	ShipToPartyID, customer_assoc
To-many / composition	camelCase plural	items	itemList, SalesOrderItems
Composition backlink	always parent	parent	header, order, toHeader
Service	PascalCase + Service	SalesOrderService	OrderSrv, ApiV2, MobileService
Service path	/api/v<n>/<kebab>	/api/v1/sales-order	/odata/SalesOrderService
Action	camelCase imperative	confirmOrder	OrderConfirm, doConfirm
Function	camelCase, read-only	calculateFreight	getFreightAndSave

Event	PascalCase past tense	OrderConfirmed	ConfirmOrder, orderConfirmedEvent
-------	-----------------------	----------------	--------------------------------------

A.2 Platform layer

Artefact	Pattern	Example
MTA ID	com.blackmores.btp.<context>	com.blackmores.btp.order
CF app (service)	blkm-btp-<context>-srv	blkm-btp-order-srv
CF app (deployer)	blkm-btp-<context>-db-deployer	blkm-btp-order-db-deployer
HDI container	blkm-btp-<context>-db	blkm-btp-order-db
XSUAA instance	blkm-btp-<context>-uaa	blkm-btp-order-uaa
Scope	\$XSAPPNAME.<Domain><Action>	\$XSAPPNAME.OrderConfirm
Role template	PascalCase job title	OrderProcessor
Role collection	BLKM_<Context><Role>	BLKM_OrderProcessor
XSUAA attribute	PascalCase, mirrors the element	SalesOrg
Destination (user)	S4_<API_NAME>	S4_API_SALES_ORDER_SRV
Destination (technical)	S4_TECH_<PURPOSE>	S4_TECH_BATCH
Event Mesh topic	blkm/<context>/<entity>/<event>/v1	blkm/order/salesorder/confirmed/v1
Environment variable	UPPER_SNAKE with unit suffix	S4_REQUEST_TIMEOUT_MS
Feature flag	FEATURE_<NAME>_ENABLED	FEATURE_ORDER_SPLIT_ENABLED

A.3 Repository layer

Artefact	Pattern	Example
----------	---------	---------

Repository	blkm-btp-<context>	blkm-btp-order
Domain model file	kebab-case plural	db/order/sales-orders.cds
Service definition	<aggregate>-service.cds	srv/sales-order-service.cds
Handler module	<aggregate>-<concern>.js	srv/handlers/sales-order-pricing.js
Pure library	kebab-case noun phrase	srv/lib/price-calculator.js
Unit test	<module>.test.js	test/unit/price-calculator.test.js
Integration test	<service>.integration.test.js	test/integration/sales-order-service.integration.test.js
Reference CSV	<namespace>-<Entity>.csv	db/data/com.blackmores.order-OrderStatuses.csv
i18n key	<domain>.<camelCaseCondition>	order.customerBlocked
Error code	UPPER_SNAKE, stable	ORDER_ABOVE_USER_LIMIT
Feature branch	feature/<TICKET>-<kebab>	feature/BLK-1234-order-split-logic
Release tag	v<semver>	v1.5.0
ADR	NNNN-kebab-title.md	docs/decisions/0007-use-event-mesh-for-stock.md

A.4 Approved abbreviations

These are the **only** abbreviations permitted in identifiers. Anything else is spelled out in full.

Abbreviation	Meaning	Abbreviation	Meaning
ID	Identifier	SKU	Stock keeping unit
UUID	Universally unique identifier	EAN / GTIN	Article / global trade item number
URL	Uniform resource locator	UOM	Unit of measure

API	Application programming interface	ETA	Estimated time of arrival
VAT	Value added tax	ABN	Australian Business Number
s4	Prefix marking a field mirrored from S/4HANA	—	—

Appendix B — Pull Request Review Checklist

A reviewer works through this list. "Looks good to me" without engaging with these points is not a review.

B.1 Model and service

- Naming conforms to Appendix A throughout — entities, elements, associations, files, services, operations.
- Service entities are projections with an explicit `excluding`; no internal or provenance fields exposed unintentionally.
- Constraints expressible in CDS are in CDS, not re-implemented in handlers.
- Compositions used only for owned children; associations for references.
- New collections carry `@cds.query.limit`; `@cds.search` restricted to indexed elements.
- Doc comments present on every new entity, service, action, and event.
- Breaking API changes, if any, are delivered on a new versioned path and flagged in the description.

B.2 Logic

- `await super.init()` is the last statement of `init()`.
- All data access via `cds.sql`; no string SQL; explicit columns; `.limit()` on unbounded reads.
- No database or remote call inside a loop.
- No module-level mutable state; handlers are stateless.
- Every promise awaited; no floating promises.
- Handlers under roughly 50 lines; business calculations extracted to pure functions in `srv/lib/`.
- Errors use `req.error()` with a stable code, an i18n key, a target, and an accurate status.

B.3 Security

- Service declares `@requires`; entities declare `@restrict` with `where` clauses where row-level filtering applies.
- No `@requires`: 'any' and no weakened authorisation left over from local testing.
- Actions granted explicitly by name.
- `xs-security.json` changes reviewed by the technical lead; role-collection impact documented per landscape.
- No secrets, tokens, `.env`, or `default-env.json` in the diff; secret scan green.
- No personal data in logs, error messages, or committed CSV/fixture files.
- Any new personal-data flow raised with the Data Privacy Officer.

B.4 Data and deployment

- Schema impact stated in the description; change is backward compatible with the currently deployed application version.
- `undeploy.json` untouched, or approved by the technical lead with backup evidence.
- CSVs contain reference data only, are complete and idempotent, and carry no personal data.
- `mta.yaml` remains landscape-neutral; landscape differences are in the relevant `.mtaext`.
- Version bumped consistently in `package.json` and `mta.yaml`.

B.5 Quality

- Lint, format, and commit-message checks green with zero warnings.
- Tests added for the new behaviour, including a negative test for every authorisation rule touched.
- A regression test accompanies every defect fix.
- Coverage floor met; no test skipped without a ticket.
- No TODO without a ticket reference; no commented-out code; no dead files.
- README, runbook, and ADRs updated where relevant.

Appendix C — Document Control

Version	Date	Author / role	Change
1.0	15 Sep 2026	Technical Lead, BTP Extension Programme	Baseline issue. CAP for Node.js backend scope: domain modelling, service layer, security, integration, HANA Cloud persistence, testing, DevOps (Azure DevOps and GitHub Actions), anti-pattern catalogue.

C.1 Review and amendment

- This document is reviewed quarterly by the programme technical lead, and out of cycle on any material change to the CAP, Node.js, or BTP baseline.
- Amendments follow the same pull-request and review process as code, in docs/standards/.
- A rule that cannot be enforced automatically or objectively assessed in review is a candidate for removal at the next review. The standard is only useful to the extent it is applied.

C.2 Placeholders to confirm

The following values are assumptions made in the absence of confirmed landscape detail, and **MUST** be verified against the Blackmores BTP landscape register before this document is issued as the project baseline.

Placeholder	Assumed value	Confirm with
Subaccount / org names	blkm-btp-dev / -qas / -prd	BTP platform team
Space names	dev, qas, prd	BTP platform team
Resource prefix	blkm-	Programme architect
Work-item prefix	BLK-	Programme PMO
Default locale	en_AU, fallback en	Business lead
Node.js LTS major	22.x	BTP platform team (CF buildpack version)
@sap/cds major	8.x	Programme architect
CI platform	Azure DevOps or GitHub Actions	Programme DevOps lead
Coverage floor	80% statements and branches	Technical lead and QA lead

BEFORE ISSUE This document was drafted as a project baseline and should be reviewed and validated by the programme technical lead and the Blackmores architecture authority before it is circulated as approved guidance or shared outside the delivery team. Library versions and platform names should be re-checked against the current SAP release notes at the time of issue.